

Complete Machine Learning CDAC Notes - 100 Percent Syllabus Coverage

Based on uploaded Machine Learning.pdf + CDAC PGCP-BDA Practical Machine Learning syllabus

Prepared for Anmol Gangwar

June 2026

Contents

1	Promise of This Version	11
2	CDAC Syllabus Coverage Checklist	12
2.1	Foundations and ML Theory	12
2.2	Session 6 and 7 Coverage	12
2.3	Session 8 and 9 Coverage	12
2.4	Session 10 and 11 Coverage	12
2.5	Session 12 Coverage	12
2.6	Session 13 and 14 Coverage	12
2.7	Session 15 and 16 Coverage	13
2.8	Session 17 to 22 Coverage	13
2.9	Session 23 and 24 Coverage	13
3	Part A - Core Notes from Uploaded PDF and CDAC Basics	14
4	How to Use These Notes	15
5	1. Artificial Intelligence, Machine Learning, Deep Learning, and Generative AI	16
5.1	1.1 Artificial Intelligence	16
5.2	1.2 Machine Learning	16
5.3	1.3 Deep Learning	16
5.4	1.4 Generative AI	17
6	2. What is Machine Learning?	17
7	3. Machine Learning Project Life Cycle	18
7.1	3.1 Basic ML Life Cycle	18
7.2	3.2 Interview-Ready ML Life Cycle	18
7.3	3.3 Example: Loan Default Prediction	18
8	4. Types of Machine Learning	19
8.1	4.1 Supervised Learning	19
8.2	4.2 Unsupervised Learning	19
8.3	4.3 Reinforcement Learning	20
9	5. ML Techniques	20
9.1	5.1 Regression / Estimation	20
9.2	5.2 Classification	20
9.3	5.3 Clustering	20
9.4	5.4 Association	21

9.5	5.5 Anomaly Detection	21
9.6	5.6 Sequence Mining	21
9.7	5.7 Dimensionality Reduction	21
9.8	5.8 Recommendation Systems	21
106.	Important ML Concepts	21
10.16.1	Features and Target	21
10.26.2	Training Data and Testing Data	22
10.36.3	Generalization	22
10.46.4	Overfitting	22
10.56.5	Underfitting	22
10.66.6	Best Fit	23
117.	Bias, Variance, and Complexity	23
11.17.1	Bias	23
11.27.2	Variance	23
11.37.3	Bias-Variance Tradeoff	23
128.	Data Preprocessing and EDA	23
12.18.1	Data Cleaning	23
12.28.2	Missing Values	24
12.38.3	Outliers	24
12.48.4	Encoding Categorical Variables	24
12.58.5	Feature Scaling	24
12.68.6	EDA - Exploratory Data Analysis	25
139.	Regression	25
13.19.1	Simple Linear Regression	26
13.29.2	Multiple Linear Regression	26
13.39.3	Linear Regression Assumptions	26
13.49.4	Multicollinearity	27
13.59.5	Residuals	27
13.69.6	RSS, TSS, and R-Squared	27
13.79.7	OLS - Ordinary Least Squares	27
13.89.8	Regression Evaluation Metrics	28
13.8.1	MAE - Mean Absolute Error	28
13.8.2	MSE - Mean Squared Error	28
13.8.3	RMSE - Root Mean Squared Error	28
13.8.4	MAPE - Mean Absolute Percentage Error	28
13.8.5	Adjusted R-Squared	28
13.99.9	Polynomial Regression	29
13.10.10	Regularization	29
13.10.1	Ridge Regression	29
13.10.2	Lasso Regression	29
13.10.3	Elastic Net	29
13.10.11	Regression Code Template	29
1410.	Cross Validation and Model Selection	30
14.110.1	Train-Test Split	30
14.210.2	K-Fold Cross Validation	30
14.310.3	Stratified K-Fold	31
14.410.4	LOOCV	31
14.510.5	Leave-P-Out CV	31
14.610.6	Time Series CV	31
14.710.7	GridSearchCV	31
14.810.8	RandomizedSearchCV	32
1511.	Classification	32

16.12. Logistic Regression	32
16.112.1 Logistic Regression Formula	32
16.212.2 Pros	33
16.312.3 Cons	33
16.412.4 Logistic Regression Code Template	33
17.13. Confusion Matrix and Classification Metrics	33
17.113.1 Confusion Matrix Terms	33
17.213.2 Accuracy	34
17.313.3 Precision	34
17.413.4 Recall / Sensitivity / TPR	34
17.513.5 Specificity	34
17.613.6 F1-Score	35
17.713.7 ROC-AUC	35
17.813.8 Log Loss	35
18.14. Support Vector Machine - SVM	35
18.114.1 Hyperplane	35
18.214.2 Support Vectors	35
18.314.3 Margin	36
18.414.4 Kernels	36
18.514.5 Slack Variable and Soft Margin	36
18.614.6 SVM Pros and Cons	36
18.714.7 SVM Code Template	36
19.15. Decision Tree	37
19.115.1 CART	37
19.215.2 Important Terms	37
19.315.3 Entropy	37
19.415.4 Information Gain	37
19.515.5 Gini Index	38
19.615.6 Decision Tree Hyperparameters	38
19.715.7 Overfitting in Decision Tree	38
19.815.8 Decision Tree Code Template	38
20.16. Random Forest	38
20.116.1 Why Random Forest Works Better Than One Tree	39
20.216.2 Bootstrapping	39
20.316.3 Feature Randomness	39
20.416.4 Pros	39
20.516.5 Cons	39
20.616.6 Random Forest Code Template	39
21.17. K-Nearest Neighbour - KNN	40
21.117.1 Working of KNN	40
21.217.2 Distance Measures	40
21.317.3 Choosing K	40
21.417.4 Pros	40
21.517.5 Cons	40
22.18. Ensemble Learning	41
22.118.1 Voting Ensemble	41
22.218.2 Bagging	41
22.318.3 Boosting	41
22.418.4 Stacking	42
23.19. Gradient Boosting, XGBoost, LightGBM, and CatBoost	42
23.119.1 Gradient Boosting Machine - GBM	42
23.219.2 XGBoost	42

23.319.3 LightGBM	42
23.419.4 CatBoost	42
2420. Naive Bayes	43
24.120.1 Bayes Theorem	43
24.220.2 Why It Is Called Naive	43
24.320.3 Use Cases	43
24.420.4 Types of Naive Bayes	43
24.4.1 Gaussian Naive Bayes	43
24.4.2 Multinomial Naive Bayes	43
24.4.3 Bernoulli Naive Bayes	43
24.520.5 Naive Bayes Code Template	43
2521. LDA and QDA	44
25.121.1 LDA - Linear Discriminant Analysis	44
25.221.2 QDA - Quadratic Discriminant Analysis	44
2622. Unsupervised Learning	44
2723. Clustering	44
27.123.1 Types of Clustering	45
27.223.2 K-Means Clustering	45
27.323.3 Elbow Method	45
27.423.4 Silhouette Score	45
27.523.5 Cluster Profiling	46
27.623.6 Fuzzy C-Means	46
27.723.7 Hierarchical Clustering	46
27.823.8 DBSCAN	46
27.923.9 K-Means Code Template	47
2824. Dimensionality Reduction	47
28.124.1 PCA - Principal Component Analysis	47
28.224.2 Kernel PCA	48
28.324.3 Random Projection	48
28.424.4 t-SNE	48
28.524.5 Autoencoder	48
2925. Association Rule Mining	49
29.125.1 Market Basket Analysis	49
29.225.2 Apriori Algorithm	49
29.325.3 Support	49
29.425.4 Confidence	49
29.525.5 Lift	49
3026. Recommendation Systems	49
30.126.1 Popularity-Based Recommendation	50
30.226.2 Content-Based Filtering	50
30.326.3 Collaborative Filtering	50
30.426.4 Hybrid Recommendation	50
3127. Anomaly Detection	50
31.127.1 Isolation Forest	51
3228. Time Series Analysis and Forecasting	51
32.128.1 Components of Time Series	51
32.228.2 Lag Features	51
32.328.3 Moving Average and Smoothing	51
32.428.4 Stationarity	51
32.528.5 ADF Test	52

32.628.6 AR, MA, ARMA, and ARIMA	52
32.6.1AR - Auto Regressive	52
32.6.2MA - Moving Average	52
32.6.3ARMA	52
32.6.4ARIMA	52
3329. Reinforcement Learning	52
33.129.1 Markov Decision Process	53
33.229.2 Q-Learning	53
3430. Deep Learning and Neural Networks	53
34.130.1 Neural Network Structure	53
34.230.2 Forward Propagation	53
34.330.3 Loss Function	53
34.430.4 Backpropagation	53
34.530.5 Activation Functions	54
34.5.1Sigmoid	54
34.5.2Tanh	54
34.5.3ReLU	54
34.5.4Softmax	54
3531. CNN - Convolutional Neural Network	54
35.131.1 Convolution Layer	54
35.231.2 ReLU Layer	54
35.331.3 Pooling Layer	54
35.431.4 Fully Connected Layer	55
3632. RNN - Recurrent Neural Network	55
3733. Deep Learning Tools	55
3834. Information Theory for ML - CDAC Add-on	55
38.134.1 Entropy	56
38.234.2 Information Gain	56
38.334.3 Cross Entropy	56
38.434.4 KL Divergence	56
38.534.5 Mutual Information	56
3935. PAC Learning - CDAC Add-on	56
4036. VC Dimension - CDAC Add-on	57
4137. Structural Risk Minimization - CDAC Add-on	57
4238. Occam's Razor - CDAC Add-on	57
4339. No Free Lunch Theorem - CDAC Add-on	57
4440. Model Stability - CDAC Add-on	58
4541. Complete Scikit-Learn Workflow	58
45.141.1 Standard ML Workflow	58
45.241.2 Complete Classification Pipeline Template	58
45.341.3 Important Mistakes to Avoid	59
4642. Lab Exam Must-Know Programs	60
4743. Which Algorithm to Use When?	60
4844. One-Line Revision	61

4945. Final Study Order for CDAC ML	61
5046. Final Checklist	62
51Part B - Expanded 100 Percent CDAC Syllabus Coverage	62
526. Information Theory for Machine Learning	63
52.16.1 Entropy	63
52.26.2 Information Gain	63
52.36.3 Cross Entropy	63
52.46.4 KL Divergence	63
52.56.5 Mutual Information	64
52.66.6 Gini Impurity	64
537. Learning Theory: PAC, VC Dimension, SRM, Occam, No Free Lunch	65
53.17.1 Generalization	65
53.27.2 PAC Learning	65
53.37.3 VC Dimension	65
53.47.4 Empirical Risk Minimization	66
53.57.5 Structural Risk Minimization	66
53.67.6 Occam's Razor	66
53.77.7 No Free Lunch Theorem	66
53.87.8 Model Stability	67
548. Data Preparation, Feature Engineering, and Pipelines	68
54.18.1 Basic Data Inspection	68
54.28.2 Missing Values	68
54.38.3 Outliers	68
54.48.4 Encoding Categorical Variables	68
54.58.5 Feature Scaling	69
54.68.6 Train-Test Split	69
54.78.7 Pipelines	70
54.88.8 Data Leakage	70
559. Cross-Validation and Hyperparameter Tuning	71
55.19.1 Why Cross-Validation?	71
55.29.2 K-Fold Cross-Validation	71
55.39.3 Stratified K-Fold	71
55.49.4 Time Series Split	71
55.59.5 GridSearchCV	71
55.69.6 RandomizedSearchCV	72
5610. Clustering, WSS, Best K, Distance Measures, PCA, Random Projection	73
56.110.1 What is Clustering?	73
56.210.2 Why Clustering is Subjective and Exploratory	73
56.310.3 K-Means Clustering	73
56.410.4 WSS / Inertia	73
56.510.5 Elbow Method for Best K	74
56.610.6 Mall Customers Lab	74
56.710.7 How to Form Number of Clusters by Observation	75
56.810.8 Cluster Profiling	75
56.910.9 Distance Measures	75
56.100.10 Impact of Scaling on Clustering	76
56.110.11 Clustering Evaluation vs Supervised Evaluation	76
56.120.12 PCA as Representation, Not Only Compression	76
56.130.13 Random Projection as Representation	76
5711. Evaluation Metrics: Classification, Regression, Ranking, and LogLoss	77
57.111.1 Confusion Matrix	77

57.211.2 Accuracy	77
57.311.3 Precision	77
57.411.4 Recall / Sensitivity	77
57.511.5 F1-Score	77
57.611.6 Specificity	78
57.711.7 ROC Curve and ROC-AUC	78
57.811.8 LogLoss	78
57.911.9 Regression Metrics	78
57.9.1MAE	78
57.9.2MSE	79
57.9.3RMSE	79
57.9.4R2 Score	79
57.9.5Adjusted R2	79
57.101.10 Metric Selection Guide	79
5812. Decision Trees and CART	80
58.112.1 What is a Decision Tree?	80
58.212.2 CART	80
58.312.3 Important Terms	80
58.412.4 Splitting Criteria	80
58.512.5 Decision Tree Hyperparameters	80
58.612.6 Overfitting in Decision Trees	81
58.712.7 Decision Tree Code	81
5913. Random Forest, GBM, Stacking, XGBoost, LightGBM, CatBoost	82
59.113.1 Ensemble Learning	82
59.213.2 Voting	82
59.313.3 Bagging	82
59.413.4 Random Forest	82
59.513.5 Boosting	83
59.613.6 Gradient Boosting Machine	83
59.713.7 XGBoost	83
59.813.8 LightGBM	84
59.913.9 CatBoost	84
59.103.10 Model Stacking	85
59.113.11 Bagging vs Boosting vs Stacking	85
6014. Session 6 and 7 Lab: Random Forest, SVM, Logistic Regression	86
6115. Bayesian Analysis and Naive Bayes	88
61.115.1 Bayesian Thinking	88
61.215.2 Naive Bayes Classifier	88
61.315.3 Probability Calculation Example	88
61.415.4 Types of Naive Bayes	88
61.515.5 Naive Bayes Code	89
61.615.6 Text Classification with MultinomialNB	89
61.715.7 Pros and Cons	89
6216. Linear Discriminant Analysis and QDA	90
62.116.1 What is LDA?	90
62.216.2 LDA Goal	90
62.316.3 LDA Assumptions	90
62.416.4 LDA vs PCA	90
62.516.5 LDA Code	90
62.616.6 QDA	90
6317. K-Nearest Neighbors Algorithm	91
63.117.1 What is KNN?	91
63.217.2 KNN Working	91

63.317.3	Choosing K	91
63.417.4	Distance Measures	91
63.517.5	Why Scaling is Important for KNN	91
63.617.6	KNN Lab Code	91
63.717.7	Find Best K	92
63.817.8	KNN Pros and Cons	92
64.18.	Regression: Linear, Polynomial, Ridge, Lasso, ElasticNet	94
64.118.1	Regression as Modeling, Not Only Curve Fitting	94
64.218.2	Simple Linear Regression	94
64.318.3	Multiple Linear Regression	94
64.418.4	Linear Regression Assumptions	94
64.518.5	Polynomial Regression	94
64.618.6	Regularization in High-Dimensional Data	94
64.718.7	Ridge Regression	95
64.818.8	Lasso Regression	95
64.918.9	ElasticNet Regression	95
64.1018.10	Bias-Variance Comparison of Regression Techniques	95
64.118.11	Regression Lab Code	95
65.19.	Logistic Regression	98
65.119.1	Why Logistic Regression?	98
65.219.2	Use Cases	98
65.319.3	Code	98
65.419.4	Pros and Cons	98
66.20.	Support Vector Machines and Air Quality Lab	99
66.120.1	SVM Basic Principle	99
66.220.2	Linear SVM	99
66.320.3	Margin Maximization as Geometric Learning	99
66.420.4	Soft Margin and Slack Variable	99
66.520.5	Kernels as Implicit Feature Transformation	99
66.620.6	Linear vs Non-Linear Decision Boundaries	100
66.720.7	Important SVM Hyperparameters	100
66.820.8	Air Quality Lab: Regression + SVM Classification	100
67.21.	Unsupervised Learning Beyond K-Means	103
67.121.1	Hierarchical Clustering	103
67.221.2	DBSCAN	103
67.321.3	Association Rule Mining	104
67.421.4	Anomaly Detection	104
68.22.	Reinforcement Learning	105
68.122.1	Important Terms	105
68.222.2	Markov Decision Process	105
68.322.3	Q-Learning	105
69.23.	Time Series: Moving Average, Holt, Holt-Winters, ACF, PACF, AR, MA, ARMA, ARIMA	107
69.123.1	What is Time Series?	107
69.223.2	Components of Time Series	107
69.323.3	Moving Average	107
69.423.4	Exponential Smoothing	107
69.523.5	Holt's Trend Method	108
69.623.6	Holt-Winters Method	108
69.723.7	Autocorrelation	108
69.823.8	ACF and PACF	109
69.923.9	Stationarity	109
69.1023.10	Differencing	109
69.1123.11	AR Model	110

69.123.12 MA Model	110
69.123.13 ARMA Model	110
69.123.14 ARIMA Model	110
69.123.15 ARMA vs ARIMA	110
69.123.16 ARIMA Lab Code	110
7024. Recommendation Systems	112
70.124.1 What is a Recommendation System?	112
70.224.2 Data Collection and Storage	112
70.324.3 Data Filtering	112
70.424.4 Popularity-Based Recommendation	112
70.524.5 Content-Based Filtering	113
70.624.6 Collaborative Filtering	113
70.724.7 User-Based Collaborative Filtering	113
70.824.8 Item-Based Collaborative Filtering	113
70.924.9 Matrix Factorization	113
70.104.10 Hybrid Recommendation	114
70.114.11 Recommendation Metrics	114
70.11.1 Precision@K	114
70.11.2 Recall@K	114
70.11.3 RMSE	114
70.11.4 Mean Reciprocal Rank	114
70.11.5 MAP@K	115
70.11.6 NDCG	115
70.124.12 Recommendation Lab Skeleton	115
70.134.13 Cold Start Problem	115
7125. Introduction to Deep Learning, TensorFlow, Keras, and Neural Networks	117
71.125.1 What is Deep Learning?	117
71.225.2 Why Deep Learning Works	117
71.325.3 Neural Network Structure	117
71.425.4 Single Layer Neural Network	117
71.525.5 Why Non-Linear Activation Functions are Essential	117
71.625.6 TensorFlow	118
71.725.7 Keras	118
71.825.8 Neural Network Applications	118
7226. Autoencoders	119
72.126.1 What is an Autoencoder?	119
72.226.2 How Autoencoders Differ from Predictive Models	119
72.326.3 Representation Learning	119
72.426.4 Undercomplete Autoencoder	119
72.526.5 Denoising Autoencoder	119
72.626.6 Autoencoder Code	119
72.726.7 Autoencoder for Anomaly Detection	120
7327. Activation Functions	121
73.127.1 Sigmoid	121
73.227.2 Tanh	121
73.327.3 ReLU	121
73.427.4 Softmax	121
73.527.5 Activation Function Lab	121
7428. Backpropagation and Weight Update with Single Training Example	123
74.128.1 Forward Propagation	123
74.228.2 Backpropagation	123
74.328.3 Gradient Descent Update Rule	123
74.428.4 Single Training Example Weight Update	123
74.528.5 Why Backpropagation is Important	124

74.628.6 Common Problems	124
7529. Deep Learning Essentials: Overfitting, Early Stopping, Dropout, L1/L2	125
75.129.1 Why Deep Networks Overfit	125
75.229.2 Early Stopping	125
75.329.3 Dropout	125
75.429.4 L1 and L2 Regularization	125
75.529.5 Regularization as Learning Constraint	126
7630. Batch Training, Mini-Batch, SGD, Momentum, Adam, Gradient Problems	127
76.130.1 Batch Training	127
76.230.2 Stochastic Gradient Descent	127
76.330.3 Mini-Batch Training	127
76.430.4 Convergence and Efficiency Comparison	127
76.530.5 Optimizers as Navigation Strategies	127
76.630.6 Classic Backpropagation with SGD	128
76.730.7 Momentum	128
76.830.8 Adam Optimizer	128
76.930.9 Faster Convergence vs Long-Term Generalization	128
76.100.10 Gradient Problems	128
76.100.11 Optimizer Lab	129
7731. CNN Using PyTorch, PyTorch vs TensorFlow, and Convolution Concept	130
77.131.1 What is PyTorch?	130
77.231.2 PyTorch vs TensorFlow	130
77.331.3 What is CNN?	130
77.431.4 Convolution as Representation Learning	130
77.531.5 Filter / Kernel	130
77.631.6 Stride and Padding	131
77.731.7 Pooling	131
77.831.8 Basic PyTorch CNN Code	131
77.931.9 PyTorch Training Loop Skeleton	132
77.101.10 PyTorch Documentation Exploration Lab	132
7832. Transfer Learning, Inception Network, and Data Augmentation	133
78.132.1 What is Transfer Learning?	133
78.232.2 Transfer Learning Strategy	133
78.332.3 Inception Network	133
78.432.4 Why Transfer Learning Helps in Data-Scarce Environments	133
78.532.5 Data Augmentation	134
78.632.6 Transfer Learning PyTorch Skeleton	134
7933. Object Detection and YOLO Algorithm	136
79.133.1 Image Classification vs Object Detection	136
79.233.2 Object Detection Output	136
79.333.3 Important Metrics for Object Detection	136
79.433.4 What is YOLO?	136
79.533.5 Why YOLO is Fast	136
79.633.6 YOLO High-Level Working	137
79.733.7 Non-Max Suppression	137
79.833.8 YOLO Lab Workflow	137
8034. Complete Lab Code Bank and Exam Patterns	139
80.134.1 Basic ML Lab Template	139
80.234.2 Classification Model Comparison Template	139
80.334.3 Complete Pipeline with ColumnTransformer	140
80.434.4 Save and Load Model	141
8135. Final Revision Tables	142

81.135.1 Algorithm Selection	142
81.235.2 Scaling Required or Not	142
81.335.3 Overfitting Control	142
81.435.4 Most Important Interview Lines	142
8236. Final CDAC Study Order	144
83End Note	144

1 Promise of This Version

This is the expanded version of the Machine Learning notes. It is built to cover the uploaded Machine Learning PDF and the CDAC PGCP-BDA Practical Machine Learning syllabus topics you shared. The previous version was useful for core revision; this version adds the missing depth, lab workflows, algorithms, metrics, deep learning, PyTorch, CNN, transfer learning, object detection, YOLO overview, time series, recommendation metrics, and all self-learning reflection points.

Important correction for lab preparation: F1-score is a classification metric. For pure regression use MAE, MSE, RMSE, R2 and Adjusted R2. If a regression value is converted into classes, then F1-score can be used on those classes.

2 CDAC Syllabus Coverage Checklist

The following checklist maps the syllabus items to where they are covered in this PDF.

2.1 Foundations and ML Theory

- Artificial Intelligence, Machine Learning, Deep Learning, Generative AI - Chapters 1 to 3.
- ML applications and project life cycle - Chapters 2 and 3.
- Supervised, Unsupervised, Reinforcement Learning - Chapters 4, 5, 21, and 22.
- Information theory: entropy, information gain, cross entropy, KL divergence, mutual information - Chapter 6 and Decision Tree chapter.
- PAC learning, VC dimension, SRM, Occam's Razor, No Free Lunch theorem - Chapter 7.
- Bias-variance, overfitting, underfitting, generalization, model stability - Chapters 7 and 10.
- Scikit-learn workflow, pipelines, preprocessing, cross-validation - Chapters 8, 9, and lab sections.

2.2 Session 6 and 7 Coverage

- Evaluation metrics: Accuracy, F1-score, ROC-AUC, LogLoss - Chapter 11.
- Regression metrics: MAE, MSE, RMSE, R2 score - Chapter 11.
- Decision Trees, CART - Chapter 12.
- Random Forest, Gradient Boosting Machines, Model Stacking - Chapter 13.
- CatBoost, LightGBM, XGBoost - Chapter 13.
- Lab: Random Forest, SVM, Logistic Regression with classification report and F1-score - Chapter 14.

2.3 Session 8 and 9 Coverage

- Bayesian analysis and Naive Bayes classifier - Chapter 15.
- Assigning probabilities and calculating results - Chapter 15.
- Linear Discriminant Analysis - Chapter 16.
- K-Nearest Neighbors Algorithm - Chapter 17.
- Lab: Implement KNN - Chapter 17.

2.4 Session 10 and 11 Coverage

- Linear Regression - Chapter 18.
- Logistic Regression - Chapter 19.
- Polynomial Regression - Chapter 18.
- Ridge, Lasso, ElasticNet Regression - Chapter 18.
- Lab: linear, Ridge, Lasso, Polynomial regression with MAE, MSE, RMSE, R2 and conclusion - Chapter 18.
- Self-learning: regression as modeling, regularization in high-dimensional data, bias-variance comparison - Chapter 18.

2.5 Session 12 Coverage

- Support Vector Machines - Chapter 20.
- Basic classification principle of SVM - Chapter 20.
- Linear and Nonlinear classification: Polynomial and RBF kernels - Chapter 20.
- Lab: Air Quality Index prediction using Linear Regression and SVM category classification - Chapter 20.
- Self-learning: margin maximization, kernel methods, linear vs non-linear boundaries - Chapter 20.

2.6 Session 13 and 14 Coverage

- Moving average, Exponential Smoothing, Holt's Trend Method, Holt-Winters - Chapter 23.
- Seasonality methods - Chapter 23.

- ACF, PACF, Auto-regression, AR, MA, ARMA, ARIMA - Chapter 23.
- Lab: autocorrelation calculation and ARMA/ARIMA explanation - Chapter 23.

2.7 Session 15 and 16 Coverage

- Recommendation Systems - Chapter 24.
- Data collection and storage, data filtering - Chapter 24.
- Collaborative filtering - Chapter 24.
- Factorization methods - Chapter 24.
- Metrics: Recall, Precision, RMSE, Mean Reciprocal Rank, MAP@K, NDCG - Chapter 24.

2.8 Session 17 to 22 Coverage

- Introduction to Deep Learning - Chapter 25.
- TensorFlow and Keras - Chapter 25.
- Autoencoders - Chapter 26.
- Neural Network applications - Chapter 25.
- Single layer neural network - Chapter 25.
- Activation functions: Sigmoid, Tanh, ReLU - Chapter 27.
- Backpropagation overview - Chapter 28.
- Early stopping, Dropout, L1 and L2 regularization - Chapter 29.
- Single training example weight update - Chapter 28.
- Batch training, mini-batch training, SGD - Chapter 30.
- Optimizers: classic backpropagation, momentum, Adam - Chapter 30.
- Gradient problems - Chapter 30.

2.9 Session 23 and 24 Coverage

- CNN using PyTorch - Chapter 31.
- Introduction to PyTorch - Chapter 31.
- PyTorch vs TensorFlow - Chapter 31.
- Convolutional concept - Chapter 31.
- Transfer learning and Inception Network - Chapter 32.
- Data augmentation - Chapter 32.
- Object detection - Chapter 33.
- YOLO Algorithm high-level overview - Chapter 33.
- Lab: PyTorch exploration and YOLO implementation workflow - Chapter 33.

3 Part A - Core Notes from Uploaded PDF and CDAC Basics

4 How to Use These Notes

These notes combine two sources:

1. The uploaded **Machine Learning.pdf** notes.
2. Your **CDAC PGCP-BDA Practical Machine Learning syllabus** topics discussed earlier.

The PDF covers many important ML topics: AI, ML, DL, Generative AI, ML life cycle, supervised learning, unsupervised learning, reinforcement learning, regression, classification, clustering, PCA, association rules, recommendation systems, anomaly detection, time series, and deep learning.

Your CDAC syllabus also expects a few theory topics that are not strongly covered in the PDF. These are added clearly in the later chapters: information theory, PAC learning, VC dimension, bias-complexity tradeoff, structural risk minimization, Occam's Razor, No Free Lunch theorem, model stability, scikit-learn workflow, model validation, cross-validation, PCA, Kernel PCA, random projections, and complete ML pipeline practice.

Study sequence:

1. First understand the basic idea.
2. Then learn the formula or process.
3. Then remember use cases.
4. Then practice the Python syntax.
5. Finally, revise the interview points.

5 1. Artificial Intelligence, Machine Learning, Deep Learning, and Generative AI

5.1 1.1 Artificial Intelligence

Artificial Intelligence, or AI, means creating machines that can imitate human intelligence and behavior.

AI systems can perform tasks such as:

- Visual perception
- Speech recognition
- Problem solving
- Reasoning
- Decision making
- Language understanding
- Creativity

Simple memory line:

AI = making machines act intelligently like humans.

Examples:

- Chatbots
- Face recognition
- Voice assistants
- Self-driving cars
- Recommendation engines
- Fraud detection systems

5.2 1.2 Machine Learning

Machine Learning, or ML, is a subset of AI. It allows computers to learn patterns from data without being explicitly programmed for every rule.

Traditional programming:

Rules + Data -> Output

Machine learning:

Data + Output -> Model learns rules

Prediction stage:

New Data + Learned Model -> Prediction

Simple memory line:

ML = learning from data.

5.3 1.3 Deep Learning

Deep Learning, or DL, is a subset of ML. It uses artificial neural networks with many layers to learn complex patterns from large amounts of data.

Simple memory line:

DL = ML using multi-layer neural networks.

Deep learning is commonly used in:

- Image recognition

- Speech recognition
- Natural language processing
- Autonomous driving
- Medical image analysis
- Generative AI

5.4 1.4 Generative AI

Generative AI is a type of AI that creates new content. It learns patterns from large data and generates text, images, code, music, audio, or video.

Hierarchy:

AI > Machine Learning > Deep Learning > Generative AI > LLMs > ChatGPT-like systems

Generative AI can work with:

- Text
- Images
- Speech
- Structured data
- 3D signals

Common tasks:

- Question answering
- Sentiment analysis
- Information extraction
- Image captioning
- Object recognition
- Instruction following
- Text generation
- Code generation

6 2. What is Machine Learning?

Machine Learning gives computers the ability to learn from experience/data without being explicitly programmed.

Basic ML flow:

Input Data -> Feature Extraction -> ML Model -> Output / Prediction

Example:

Input: Customer age, salary, previous purchases

Model: Classification model

Output: Will the customer buy? Yes/No

Applications:

- Amazon/Flipkart recommendation systems
- Loan default prediction
- Telecom customer plan analysis
- Chatbots
- Face recognition
- Spam detection
- Disease prediction
- Sales forecasting
- Fraud detection

7 3. Machine Learning Project Life Cycle

7.1 3.1 Basic ML Life Cycle

The basic ML life cycle is:

1. Collecting data
2. Data wrangling / cleaning
3. Data analysis / EDA
4. Train algorithm
5. Test algorithm
6. Deployment

7.2 3.2 Interview-Ready ML Life Cycle

For real projects and interviews, explain the life cycle like this:

1. Business objective
2. Data collection
3. Data cleaning
4. Exploratory Data Analysis (EDA)
5. Feature engineering
6. Feature selection
7. Train-test split
8. Model training
9. Model evaluation
10. Hyperparameter tuning
11. Final model selection
12. Model deployment
13. Monitoring and retraining

7.3 3.3 Example: Loan Default Prediction

Business objective:

Predict whether a customer will default on a loan.

Data collection:

Collect customer age, income, loan amount, credit history, employment status.

Data cleaning:

Handle missing values, duplicate rows, wrong data types, outliers.

EDA:

Check which features are related to default: income, credit history, loan amount.

Model training:

Train Logistic Regression, Decision Tree, Random Forest, SVM.

Evaluation:

Use accuracy, precision, recall, F1-score, ROC-AUC.

Deployment:

Expose the model using Flask/FastAPI or use it in a dashboard.

8 4. Types of Machine Learning

Machine Learning is mainly divided into:

1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

8.1 4.1 Supervised Learning

Supervised learning uses labelled data. This means both input and correct output are known during training.

Example:

Input: Age, Salary, Experience
Output: Purchased = Yes/No

The model learns from historical input-output pairs.

Supervised learning has two main types:

Regression -> predicts continuous numerical values
Classification -> predicts categories/classes

Examples:

Problem	Type
House price prediction	Regression
Salary prediction	Regression
Sales prediction	Regression
Spam/ham email	Classification
Loan approved/rejected	Classification
Disease/no disease	Classification

8.2 4.2 Unsupervised Learning

Unsupervised learning uses unlabelled data. The model does not know the correct output. It discovers hidden patterns by itself.

Example:

Input: Customer age, income, spending score
Output: Groups of similar customers

Main techniques:

- Clustering
- Dimensionality reduction
- Association rule mining
- Recommendation systems
- Anomaly detection

Examples:

- Customer segmentation
- Product grouping
- Market basket analysis
- Fraud pattern discovery
- Document clustering

8.3 4.3 Reinforcement Learning

Reinforcement Learning, or RL, is learning through trial and error.

The model is called an agent. The agent performs actions in an environment and receives rewards or penalties.

Example:

```
Agent: Robot
Environment: Room
Action: Move left/right/forward/backward
Reward: +10 for reaching goal, -1 for wrong move
```

Important terms:

- Agent
- Environment
- State
- Action
- Reward
- Policy
- Value
- Q-value

9 5. ML Techniques

The uploaded PDF lists important ML techniques.

9.1 5.1 Regression / Estimation

Regression predicts continuous values.

Examples:

- Predict salary
- Predict house price
- Predict sales
- Predict crop yield
- Predict delivery time

9.2 5.2 Classification

Classification predicts class/category labels.

Examples:

- Spam or not spam
- Fraud or not fraud
- Loan approved or rejected
- Customer churn or not churn

9.3 5.3 Clustering

Clustering finds groups in data.

Example:

```
Group customers into low, medium, and high spending segments.
```

9.4 5.4 Association

Association finds frequent co-occurring items/events.

Example:

People who buy bread also buy butter.

9.5 5.5 Anomaly Detection

Anomaly detection finds abnormal cases.

Examples:

- Fraud transaction
- Network attack
- Machine failure
- Medical abnormality

9.6 5.6 Sequence Mining

Sequence mining predicts next events.

Examples:

- Clickstream prediction
- Next product prediction
- User behavior sequence

9.7 5.7 Dimensionality Reduction

Dimensionality reduction reduces number of features while keeping important information.

Examples:

- PCA
- t-SNE
- ICA
- SVD
- Autoencoder

9.8 5.8 Recommendation Systems

Recommendation systems suggest items.

Examples:

- Netflix movie recommendation
- YouTube video recommendation
- Amazon product recommendation

10 6. Important ML Concepts

10.1 6.1 Features and Target

In ML, input columns are called features, and the output column is called target.

X = features / independent variables
y = target / dependent variable

Example:

X = Age, Salary, Experience
y = Purchased

10.2 6.2 Training Data and Testing Data

Training data is used to teach the model. Testing data is used to check model performance on unseen data.

Training data -> model learns
Testing data -> model is evaluated

10.3 6.3 Generalization

Generalization means the model performs well on unseen data.

A model should not only memorize training data. It should learn real patterns.

Good generalization = good performance on train and test data.

10.4 6.4 Overfitting

Overfitting happens when a model performs very well on training data but poorly on testing data.

Signs:

Training score high
Testing score low

Reason:

Model is too complex and learns noise.

How to reduce overfitting:

- Use more data
- Use cross-validation
- Use regularization
- Use simpler model
- Prune decision tree
- Reduce features
- Use dropout in neural networks

10.5 6.5 Underfitting

Underfitting happens when a model performs poorly on both training and testing data.

Signs:

Training score low
Testing score low

Reason:

Model is too simple and cannot learn the pattern.

How to reduce underfitting:

- Use a more complex model
- Add useful features
- Reduce regularization

- Train longer
- Use polynomial features

10.6 6.6 Best Fit

Best fit means the model performs well on both training and testing data.

Training score high
Testing score high

11 7. Bias, Variance, and Complexity

This topic is important for CDAC theory.

11.1 7.1 Bias

Bias is error caused by wrong assumptions or an overly simple model.

High bias causes underfitting.

Example:

Using a straight line for highly curved data.

11.2 7.2 Variance

Variance is error caused by too much sensitivity to training data.

High variance causes overfitting.

Example:

A very deep decision tree memorizes training examples.

11.3 7.3 Bias-Variance Tradeoff

A good model balances bias and variance.

Model Type	Bias	Variance	Problem
Too simple	High	Low	Underfitting
Too complex	Low	High	Overfitting
Balanced	Medium	Medium	Good fit

Memory line:

Underfitting = high bias
Overfitting = high variance
Good model = balanced bias and variance

12 8. Data Preprocessing and EDA

12.1 8.1 Data Cleaning

Data cleaning means making raw data usable.

Common tasks:

- Remove duplicates
- Handle missing values
- Correct data types
- Handle outliers
- Fix inconsistent categories
- Remove irrelevant columns

12.2 8.2 Missing Values

Missing values can be handled using:

- Drop rows
- Drop columns
- Mean imputation
- Median imputation
- Mode imputation
- Forward fill / backward fill
- Model-based imputation

Rules:

Numerical column -> mean or median
 Categorical column -> mode
 Time series -> forward fill/backward fill carefully

12.3 8.3 Outliers

Outliers are extreme values.

Detection methods:

- Boxplot
- Z-score
- IQR method

IQR rule:

$IQR = Q3 - Q1$
 $Lower\ Bound = Q1 - 1.5 * IQR$
 $Upper\ Bound = Q3 + 1.5 * IQR$

12.4 8.4 Encoding Categorical Variables

ML models need numbers, not text.

Common encoding techniques:

- Label Encoding
- One-Hot Encoding
- Ordinal Encoding

Example:

Male -> 1
 Female -> 0

For unordered categories like city or product type, One-Hot Encoding is usually better.

12.5 8.5 Feature Scaling

Feature scaling brings columns to similar ranges.

Important for:

- KNN
- SVM
- Logistic Regression
- Linear Regression
- PCA
- K-Means
- Neural Networks

Usually not required for:

- Decision Tree
- Random Forest
- XGBoost
- CatBoost

Types:

StandardScaler -> mean 0, standard deviation 1

MinMaxScaler -> values between 0 and 1

RobustScaler -> uses median and IQR, good for outliers

12.6 8.6 EDA - Exploratory Data Analysis

EDA means understanding the data before modeling.

Basic EDA checklist:

1. df.head()
2. df.tail()
3. df.shape
4. df.info()
5. df.describe()
6. df.isnull().sum()
7. df.duplicated().sum()
8. value_counts()
9. correlation matrix
10. charts and visualizations

Common charts:

- Histogram
- Bar chart
- Boxplot
- Scatter plot
- Line chart
- Heatmap

13 9. Regression

Regression predicts continuous numerical values.

Examples:

- Salary prediction
- House price prediction
- Sales forecasting
- Crop yield prediction
- Delivery time prediction

Regression algorithms covered in the PDF:

- Simple Linear Regression
- Multiple Linear Regression
- Ordinary Least Squares
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Elastic Net
- Time Series Forecasting

13.1 9.1 Simple Linear Regression

Simple Linear Regression is used when there is one independent variable and one dependent variable.

Formula:

$$y = b_0 + b_1x$$

Where:

y = predicted output
 b₀ = intercept
 b₁ = slope/coefficient
 x = input feature

Example:

Experience -> Salary

Interpretation:

If b₁ is positive, y increases when x increases.
 If b₁ is negative, y decreases when x increases.

13.2 9.2 Multiple Linear Regression

Multiple Linear Regression is used when there are multiple input features.

Formula:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

Example:

House Price = b₀ + b₁(Size) + b₂(Rooms) + b₃(LocationScore)

13.3 9.3 Linear Regression Assumptions

Important assumptions:

1. Linear relationship between input and output
2. No multicollinearity
3. Normality of residuals
4. Homoscedasticity
5. No autocorrelation of errors

Simple meaning:

Assumption	Meaning
Linearity	Input and output should roughly follow a line

Assumption	Meaning
No multicollinearity	Features should not be highly correlated with each other
Normal residuals	Errors should be normally distributed
Homoscedasticity	Error spread should be almost constant
No autocorrelation	Errors should not depend on previous errors

13.4 9.4 Multicollinearity

Multicollinearity occurs when independent variables are highly correlated with each other.

Problem:

It becomes difficult to know the individual effect of each feature.

Detection methods:

- Correlation matrix
- VIF
- Eigenvalues

VIF rules:

VIF < 5 -> good

VIF 5 to 10 -> inspect

VIF > 10 -> high multicollinearity; consider removing feature

13.5 9.5 Residuals

Residual means error.

Residual = Actual value - Predicted value

If residual is small, prediction is good.

13.6 9.6 RSS, TSS, and R-Squared

RSS means Residual Sum of Squares.

RSS = sum of squared residuals

High RSS means bad model.

TSS means Total Sum of Squares.

R-squared:

$R^2 = 1 - \text{RSS}/\text{TSS}$

Interpretation:

R² close to 1 -> good model

R² close to 0 -> weak model

13.7 9.7 OLS - Ordinary Least Squares

OLS finds the best regression line by minimizing squared errors.

Goal:

Minimize RSS

Important OLS summary terms:

- R-squared
- Adjusted R-squared
- F-statistic
- Prob(F-statistic)
- Coefficient
- p-value
- Standard error

Rules:

Prob(F-statistic) < 0.05 -> overall model is significant
p-value < 0.05 -> feature is significant
Higher R-squared -> better fit

13.8 9.8 Regression Evaluation Metrics

Important regression metrics:

- MAE
- MSE
- RMSE
- MAPE
- R-squared
- Adjusted R-squared

13.8.1 MAE - Mean Absolute Error

MAE = average of absolute errors

Easy to understand because it is in the same unit as the target.

13.8.2 MSE - Mean Squared Error

MSE = average of squared errors

It penalizes large errors more.

13.8.3 RMSE - Root Mean Squared Error

RMSE = square root of MSE

It is in the same unit as the target.

13.8.4 MAPE - Mean Absolute Percentage Error

MAPE = average percentage error

Useful when you want error in percentage.

13.8.5 Adjusted R-Squared

Adjusted R-squared is better when multiple features are used because it penalizes unnecessary features.

13.9 9.9 Polynomial Regression

Polynomial Regression models non-linear relationships using polynomial features.

Formula:

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$$

Memory:

Linear Regression = straight line

Polynomial Regression = curved line

Use when:

The relationship is curved, not straight.

13.10 9.10 Regularization

Regularization reduces overfitting by controlling coefficient magnitude.

Types:

Ridge Regression = L2 regularization

Lasso Regression = L1 regularization

Elastic Net = L1 + L2

13.10.1 Ridge Regression

Ridge adds penalty on squared coefficients.

$$\text{Loss} = \text{MSE} + \lambda * \sum(\text{coefficient}^2)$$

Effect:

- Reduces coefficient values
- Usually does not make coefficients zero
- Good when many features are useful

13.10.2 Lasso Regression

Lasso adds penalty on absolute coefficients.

$$\text{Loss} = \text{MSE} + \lambda * \sum(|\text{coefficient}|)$$

Effect:

- Can make coefficients exactly zero
- Works like automatic feature selection

13.10.3 Elastic Net

Elastic Net combines Ridge and Lasso.

Use when:

There are many correlated features.

13.11 9.11 Regression Code Template

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

X = df[["feature1", "feature2"]]
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2:", r2_score(y_test, y_pred))

```

14 10. Cross Validation and Model Selection

Cross-validation checks whether a model performs well on different parts of the data.

14.1 10.1 Train-Test Split

Basic method:

Training data -> model learns
 Testing data -> model is evaluated

Correct syntax:

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

Common mistake to avoid:

Wrong: X_train, y_train, X_test, y_test
 Correct: X_train, X_test, y_train, y_test

14.2 10.2 K-Fold Cross Validation

K-Fold divides data into K parts. Each part becomes test data once.

Example:

K = 5
 Train on 4 folds
 Test on 1 fold
 Repeat 5 times
 Average the score

14.3 10.3 Stratified K-Fold

Used mainly for classification. It maintains class ratio in every fold.

Example:

```
Original data: 70% Yes, 30% No
Each fold: approximately 70% Yes, 30% No
```

Useful for imbalanced classification.

14.4 10.4 LOOCV

Leave-One-Out Cross Validation uses one row as test data each time.

```
If there are 100 rows, model trains 100 times.
```

Accurate but slow.

14.5 10.5 Leave-P-Out CV

Leave-P-Out leaves P samples for testing each time.

More expensive than LOOCV if P is large.

14.6 10.6 Time Series CV

Time series data must preserve time order.

Correct approach:

```
Train on past -> test on future
```

Never randomly shuffle time series data.

14.7 10.7 GridSearchCV

GridSearchCV tries all parameter combinations.

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

params = {
    "max_depth": [3, 5, 10],
    "criterion": ["gini", "entropy"]
}

model = DecisionTreeClassifier(random_state=42)
grid = GridSearchCV(model, params, cv=5, scoring="accuracy")
grid.fit(X_train, y_train)

print(grid.best_params_)
print(grid.best_score_)
```

14.8 10.8 RandomizedSearchCV

RandomizedSearchCV tries random parameter combinations.

Use when:

Parameter grid is very large.

15 11. Classification

Classification predicts categories/classes.

Examples:

- Spam or not spam
- Fraud or not fraud
- Loan approved or rejected
- Disease or no disease
- Customer churn or no churn
- Fake news or real news

Important classification algorithms:

- Logistic Regression
- KNN
- SVM
- Decision Tree
- Random Forest
- Naive Bayes
- LDA
- QDA
- XGBoost
- CatBoost
- AdaBoost
- ANN

16 12. Logistic Regression

Logistic Regression is a supervised classification algorithm.

It is commonly used for binary classification.

Examples:

- Spam/ham email
- Loan defaulter/non-defaulter
- Disease/no disease
- Customer churn/no churn

16.1 12.1 Logistic Regression Formula

Linear part:

$$z = wX + b$$

Sigmoid function:

$$p = 1 / (1 + e^{(-z)})$$

Decision rule:


```
If p >= 0.5 -> class 1
If p < 0.5 -> class 0
```

16.2 12.2 Pros

- Easy to implement
- Easy to interpret
- Good baseline model
- Works well for linear decision boundaries
- Less prone to overfitting on low-dimensional data

16.3 12.3 Cons

- Sensitive to outliers
- Struggles with complex non-linear relationships
- Can overfit high-dimensional data
- Affected by class imbalance
- Needs enough data

16.4 12.4 Logistic Regression Code Template

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

X = df[["feature1", "feature2"]]
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

17 13. Confusion Matrix and Classification Metrics

17.1 13.1 Confusion Matrix Terms

Term	Meaning
TP	Actual positive, predicted positive
TN	Actual negative, predicted negative

Term	Meaning
FP	Actual negative, predicted positive
FN	Actual positive, predicted negative

Example for disease prediction:

TP: Patient has disease and model says disease
 TN: Patient does not have disease and model says no disease
 FP: Patient does not have disease but model says disease
 FN: Patient has disease but model says no disease

17.2 13.2 Accuracy

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

Use when classes are balanced.

17.3 13.3 Precision

Precision = $TP / (TP + FP)$

Meaning:

Out of predicted positives, how many are actually positive?

Use when false positives are costly.

Example:

Spam detection: wrongly marking important email as spam is bad.

17.4 13.4 Recall / Sensitivity / TPR

Recall = $TP / (TP + FN)$

Meaning:

Out of actual positives, how many did the model catch?

Use when false negatives are costly.

Example:

Cancer detection: missing a real patient is dangerous.

17.5 13.5 Specificity

Specificity = $TN / (TN + FP)$

Meaning:

Out of actual negatives, how many were correctly predicted negative?

17.6 13.6 F1-Score

$$F1 = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$$

Use when data is imbalanced.

17.7 13.7 ROC-AUC

ROC curve compares:

True Positive Rate vs False Positive Rate

AUC interpretation:

AUC close to 1.0 -> very good model

AUC around 0.5 -> random model

17.8 13.8 Log Loss

Log Loss measures probability prediction error.

Use when:

You care about predicted probabilities, not only class labels.

18 14. Support Vector Machine - SVM

SVM is a supervised classification algorithm. It finds the best boundary between classes.

Important terms:

- Hyperplane
- Margin
- Support vectors
- Kernel
- Soft margin
- Slack variable

18.1 14.1 Hyperplane

In 2D, a hyperplane is a line.

$$ax + by + c = 0$$

In n dimensions:

$$w^T x + b = 0$$

Classification:

$$w^T x + b > 0 \rightarrow \text{Class A}$$

$$w^T x + b < 0 \rightarrow \text{Class B}$$

18.2 14.2 Support Vectors

Support vectors are the nearest data points to the hyperplane.

They decide the position of the boundary.

Memory line:

SVM depends mainly on boundary points.

18.3 14.3 Margin

Margin is the distance between the hyperplane and nearest class points.

Best hyperplane:

Maximum margin hyperplane

18.4 14.4 Kernels

Kernels help SVM handle non-linear data.

Common kernels:

- Linear kernel
- Polynomial kernel
- RBF kernel
- Sigmoid kernel

Use:

Kernel	Use
Linear	Linearly separable data
Polynomial	Curved boundaries
RBF	Complex non-linear data
Sigmoid	Neural-network-like boundary

18.5 14.5 Slack Variable and Soft Margin

Real data is often not perfectly separable. Slack variables allow some errors.

Slack = 0 -> correctly classified and outside margin

$0 < \text{Slack} < 1$ -> correct but inside margin

Slack > 1 -> misclassified

18.6 14.6 SVM Pros and Cons

Pros:

- Works well with small datasets
- Good for high-dimensional data
- Effective when there is clear margin
- Can handle non-linear data using kernels

Cons:

- Slow for large datasets
- Not good for noisy overlapping data
- Needs proper kernel and parameter tuning

18.7 14.7 SVM Code Template

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC(kernel="linear"))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

```

19 15. Decision Tree

Decision Tree is a supervised algorithm used for classification and regression.

It has a tree-like structure:

Root node -> Decision nodes -> Leaf nodes

19.1 15.1 CART

CART means Classification and Regression Tree.

It can be used for:

- Classification
- Regression

19.2 15.2 Important Terms

Term	Meaning
Root node	First/main decision node
Decision node	Node where data is split
Leaf node	Final output node
Branch	Connection between nodes
Sub-tree	Smaller tree inside main tree

19.3 15.3 Entropy

Entropy measures impurity or disorder.

Entropy = 0 -> pure node

High entropy -> mixed node

Formula:

$$\text{Entropy}(S) = - \sum (p_i * \log_2(p_i))$$

19.4 15.4 Information Gain

Information Gain measures reduction in entropy after a split.

Information Gain = Entropy(parent) - Weighted Entropy(children)

Best split:

Feature with highest information gain

19.5 15.5 Gini Index

Gini measures impurity.

$$\text{Gini} = 1 - \sum(p_i^2)$$

Low Gini is better.

19.6 15.6 Decision Tree Hyperparameters

Important hyperparameters:

- criterion
- max_depth
- min_samples_split
- min_samples_leaf
- max_features

Meaning:

Parameter	Meaning
criterion	gini or entropy
max_depth	maximum depth of tree
min_samples_split	minimum samples needed to split node
min_samples_leaf	minimum samples required in leaf
max_features	number of features considered for split

19.7 15.7 Overfitting in Decision Tree

Decision trees can easily overfit.

Control overfitting by:

- Reducing max_depth
- Increasing min_samples_leaf
- Increasing min_samples_split
- Pruning
- Using Random Forest

19.8 15.8 Decision Tree Code Template

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

20 16. Random Forest

Random Forest is an ensemble of many decision trees.

Basic idea:

Many decision trees -> voting/averaging -> final prediction

For classification:

Final output = majority vote

For regression:

Final output = average prediction

20.1 16.1 Why Random Forest Works Better Than One Tree

A single decision tree can overfit. Random Forest reduces overfitting by training many trees on different samples and features.

20.2 16.2 Bootstrapping

Bootstrapping means sampling rows with replacement.

Original data -> random samples -> different trees

20.3 16.3 Feature Randomness

Each tree sees only a random subset of features during splitting.

This makes trees different from each other.

20.4 16.4 Pros

- Reduces overfitting
- Works well on many datasets
- Handles non-linear relationships
- Gives feature importance
- Can handle classification and regression

20.5 16.5 Cons

- Less interpretable than one tree
- Slower than one tree
- Needs more memory

20.6 16.6 Random Forest Code Template

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

21 17. K-Nearest Neighbour - KNN

KNN is a supervised algorithm used for classification and regression.

KNN is non-parametric. It does not learn a fixed equation. It stores training data and compares new points with old points.

21.1 17.1 Working of KNN

Steps:

1. Choose K
2. Calculate distance from new point to all training points
3. Select K nearest points
4. Classification: majority vote
5. Regression: average value

Example:

K = 3
Nearest classes: Yes, Yes, No
Prediction = Yes

21.2 17.2 Distance Measures

Common distance measures:

- Euclidean distance
- Manhattan distance
- Minkowski distance

21.3 17.3 Choosing K

Small K:

More sensitive to noise
Can overfit

Large K:

Smoother boundary
Can underfit

21.4 17.4 Pros

- Simple
- No training time
- Good for small datasets
- Can work for classification and regression

21.5 17.5 Cons

- Slow prediction for large data
- Sensitive to feature scaling
- Sensitive to irrelevant features
- Needs correct K value

Important:

Always scale data before KNN.

22 18. Ensemble Learning

Ensemble Learning combines multiple models to make a stronger model.

Main types:

- Voting
- Bagging
- Boosting
- Stacking

22.1 18.1 Voting Ensemble

Different models vote for the final answer.

Example:

```
Logistic Regression -> Yes
SVM -> Yes
KNN -> No
Final -> Yes
```

22.2 18.2 Bagging

Bagging trains the same algorithm on different random samples.

Example:

```
Random Forest = Bagging + Decision Trees
```

Benefit:

Reduces variance and overfitting.

22.3 18.3 Boosting

Boosting trains models sequentially. Each new model focuses on previous mistakes.

Examples:

- AdaBoost
- Gradient Boosting Machine
- XGBoost
- LightGBM
- CatBoost

Benefit:

Usually gives high accuracy on tabular data.

Risk:

Can overfit if not tuned.

22.4 18.4 Stacking

Stacking combines predictions from multiple base models using a final meta-model.

Example:

Base models: Logistic Regression, Random Forest, SVM
Meta-model: Logistic Regression

23 19. Gradient Boosting, XGBoost, LightGBM, and CatBoost

23.1 19.1 Gradient Boosting Machine - GBM

GBM combines weak learners, usually decision trees.

Main elements:

- Loss function
- Weak learners
- Additive model

Simple process:

First tree predicts.
Second tree learns first tree's errors.
Third tree learns remaining errors.
Final model combines all trees.

23.2 19.2 XGBoost

XGBoost means Extreme Gradient Boosting.

Important features:

- Regularization
- Tree pruning
- Handles missing values
- Fast training
- Parallel processing
- Supports classification, regression, and ranking

Use when:

You want high performance on structured/tabular data.

23.3 19.3 LightGBM

LightGBM is a fast gradient boosting framework.

Use when:

- Dataset is large
- Training speed is important
- Data is structured/tabular

23.4 19.4 CatBoost

CatBoost is a boosting algorithm that handles categorical variables well.

Use when:

- Dataset has many categorical columns

- You want strong performance with less encoding effort

24 20. Naive Bayes

Naive Bayes is a probabilistic classification algorithm based on Bayes theorem.

24.1 20.1 Bayes Theorem

$$P(H|E) = P(E|H) * P(H) / P(E)$$

Meaning:

$$\text{Posterior} = \text{Likelihood} * \text{Prior} / \text{Evidence}$$

24.2 20.2 Why It Is Called Naive

It assumes features are independent of each other.

This assumption is often not fully true, but the algorithm still works well in many cases.

24.3 20.3 Use Cases

- Spam filtering
- News categorization
- Sentiment analysis
- Document classification
- Weather prediction
- Medical diagnosis

24.4 20.4 Types of Naive Bayes

24.4.1 Gaussian Naive Bayes

Used for continuous numerical data.

Assumes features follow normal distribution.

24.4.2 Multinomial Naive Bayes

Used for count/frequency data.

Good for text classification.

24.4.3 Bernoulli Naive Bayes

Used for binary features.

Example:

Word present = 1
Word absent = 0

24.5 20.5 Naive Bayes Code Template

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

25 21. LDA and QDA

25.1 21.1 LDA - Linear Discriminant Analysis

LDA is a classification and dimensionality reduction method.

It finds a linear boundary between classes.

Use:

- Classification
- Feature projection
- Dimensionality reduction

Assumptions:

- Input features are numerical
- Data is roughly normally distributed
- Classes have similar covariance

25.2 21.2 QDA - Quadratic Discriminant Analysis

QDA is similar to LDA but allows curved boundaries.

Difference:

```
LDA -> linear boundary
QDA -> quadratic/curved boundary
```

Use QDA when class covariance is different.

26 22. Unsupervised Learning

Unsupervised learning works with unlabelled data.

Main tasks:

- Clustering
- Dimensionality reduction
- Association rule mining
- Recommendation systems
- Anomaly detection

27 23. Clustering

Clustering groups similar data points.

Goal:

```
High similarity within cluster
High difference between clusters
```

Use cases:

- Customer segmentation
- Retail analysis
- Banking segmentation
- Insurance risk groups
- Document grouping
- Fraud pattern discovery

27.1 23.1 Types of Clustering

- Exclusive clustering
- Overlapping clustering
- Hierarchical clustering
- Centroid-based clustering
- Connectivity-based clustering
- Distribution-based clustering
- Density-based clustering

27.2 23.2 K-Means Clustering

K-Means divides data into K clusters.

Steps:

1. Choose K
2. Randomly initialize K centroids
3. Assign each point to nearest centroid
4. Move centroid to mean of assigned points
5. Repeat until centroids stop changing

Important:

K must be chosen before training.

27.3 23.3 Elbow Method

Elbow method helps choose K.

It plots K vs WCSS/inertia.

Choose the point where error reduction starts slowing down.

Memory:

Elbow point = good K

27.4 23.4 Silhouette Score

Silhouette score measures clustering quality.

Range:

-1 to +1

Meaning:

+1 -> good clustering
 0 -> overlapping clusters
 -1 -> wrong clustering

27.5 23.5 Cluster Profiling

After clustering, describe each cluster.

Example:

Cluster 0: young customers, high spending
Cluster 1: older customers, low spending
Cluster 2: middle age, medium spending

This is important for business interpretation.

27.6 23.6 Fuzzy C-Means

In fuzzy clustering, one point can belong partly to multiple clusters.

Example:

Customer A: 70% Cluster 1, 30% Cluster 2

Useful when boundaries are not clear.

27.7 23.7 Hierarchical Clustering

Hierarchical clustering builds a tree-like structure called dendrogram.

Types:

Agglomerative -> bottom-up
Divisive -> top-down

Agglomerative process:

1. Each point starts as its own cluster
2. Nearest clusters merge
3. Continue until one big cluster
4. Cut dendrogram to choose number of clusters

Linkage methods:

- Single linkage
- Complete linkage
- Average linkage
- Ward linkage

27.8 23.8 DBSCAN

DBSCAN is density-based clustering.

Important terms:

- eps
- min_samples
- core point
- border point
- noise point

Good for:

- Irregular cluster shapes
- Noise/outlier detection

Not good when:

- Clusters have very different densities

27.9 23.9 K-Means Code Template

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = model.fit_predict(X_scaled)

df["Cluster"] = clusters
print(df.groupby("Cluster").mean(numeric_only=True))
```

28 24. Dimensionality Reduction

Dimensionality reduction reduces the number of features while preserving important information.

Why needed?

- Too many features
- Slow model
- Overfitting risk
- Difficult visualization
- Multicollinearity

Methods:

- PCA
- Kernel PCA
- Random Projection
- t-SNE
- ICA
- SVD
- Autoencoder

28.1 24.1 PCA - Principal Component Analysis

PCA converts correlated features into new independent features called principal components.

Goal:

Reduce dimensions while retaining maximum variance.

Steps:

1. Standardize data
2. Compute covariance matrix
3. Find eigenvalues and eigenvectors
4. Select top principal components
5. Transform data

Use cases:

- Visualization
- Feature reduction
- Noise reduction

- Remove multicollinearity
- Speed up ML models

Limitations:

- Less interpretable
- Works best with linear relationships
- Sensitive to scaling
- May lose information

Important:

Always scale data before PCA.

28.2 24.2 Kernel PCA

Kernel PCA is used for non-linear dimensionality reduction.

Difference:

PCA -> linear
Kernel PCA -> non-linear

Common kernels:

- RBF
- Polynomial
- Sigmoid

Use when data has curved/non-linear structure.

28.3 24.3 Random Projection

Random Projection reduces dimensions using random matrices.

Comparison:

PCA -> more accurate but slower
Random Projection -> faster but approximate

Use when:

- Data is very high-dimensional
- Speed is important
- Approximation is acceptable

28.4 24.4 t-SNE

t-SNE is mainly used for visualization.

Use:

Visualize high-dimensional data in 2D or 3D.

Usually not used as a preprocessing step before model training.

28.5 24.5 Autoencoder

Autoencoder is a neural network used for compression and reconstruction.

Flow:

Input -> Encoder -> Compressed representation -> Decoder -> Reconstructed output

29 25. Association Rule Mining

Association Rule Mining finds relationships between items.

Example:

People who buy bread also buy butter.

29.1 25.1 Market Basket Analysis

Market Basket Analysis is used in retail to find items frequently bought together.

Example rule:

{Bread, Butter} -> {Milk}

29.2 25.2 Apriori Algorithm

Apriori finds frequent itemsets and generates association rules.

Basic idea:

If an itemset is frequent, its subsets are also frequent.

29.3 25.3 Support

Support tells how frequently an itemset appears.

$\text{Support}(A) = \text{Transactions containing } A / \text{Total transactions}$

29.4 25.4 Confidence

Confidence tells how often B is bought when A is bought.

$\text{Confidence}(A \rightarrow B) = \text{Support}(A \text{ and } B) / \text{Support}(A)$

29.5 25.5 Lift

Lift tells whether A and B are truly associated.

$\text{Lift} = \text{Confidence}(A \rightarrow B) / \text{Support}(B)$

Interpretation:

Lift > 1 -> positive association
Lift = 1 -> no association
Lift < 1 -> negative association

30 26. Recommendation Systems

Recommendation systems suggest items to users.

Examples:

- Netflix recommends movies
- YouTube recommends videos
- Amazon recommends products
- Spotify recommends songs

30.1 26.1 Popularity-Based Recommendation

Recommends most popular items.

Example:

```
Top trending movies  
Top-selling products
```

No personalization.

30.2 26.2 Content-Based Filtering

Recommends similar items based on item features.

Example:

```
User watched action movies -> recommend more action movies
```

Uses:

- Item metadata
- TF-IDF
- Cosine similarity

30.3 26.3 Collaborative Filtering

Uses user behavior patterns.

Example:

```
Users similar to you liked this item, so you may like it too.
```

Types:

- User-based collaborative filtering
- Item-based collaborative filtering

30.4 26.4 Hybrid Recommendation

Combines multiple recommendation methods.

Example:

```
Netflix and YouTube use hybrid recommendation systems.
```

31 27. Anomaly Detection

Anomaly detection finds unusual patterns.

Examples:

- Fraud transaction
- Network attack
- Machine failure
- Medical abnormality
- Server error spike

Methods:

- Statistical methods
- Distance-based methods
- Clustering-based methods

- Isolation Forest
- Autoencoder-based anomaly detection

31.1 27.1 Isolation Forest

Isolation Forest isolates anomalies faster than normal points.

Simple idea:

Anomalies are few and different, so they are easier to isolate.

32 28. Time Series Analysis and Forecasting

Time series data is ordered by time.

Examples:

- Daily sales
- Stock price
- Temperature
- Electricity consumption
- Website traffic

32.1 28.1 Components of Time Series

Trend -> long-term movement
 Seasonality -> repeating pattern
 Noise -> random variation

Example:

Ice cream sales increase every summer -> seasonality

32.2 28.2 Lag Features

Lag means previous value.

$Y(t-1)$ = previous time value
 $Y(t-7)$ = value 7 time periods ago

32.3 28.3 Moving Average and Smoothing

Moving average reduces noise.

Types:

- Simple moving average
- Exponential smoothing
- Double exponential smoothing
- Holt-Winters method

32.4 28.4 Stationarity

A time series is stationary if its statistical properties do not change over time.

Stationary means:

- Mean is almost constant

- Variance is almost constant
- No strong trend/seasonality

32.5 28.5 ADF Test

ADF means Augmented Dickey-Fuller test.

Rule:

p-value < 0.05 -> stationary
p-value >= 0.05 -> not stationary

32.6 28.6 AR, MA, ARMA, and ARIMA

32.6.1 AR - Auto Regressive

Future depends on past values.

32.6.2 MA - Moving Average

Future depends on past errors/shocks.

32.6.3 ARMA

Combination of AR and MA.

32.6.4 ARIMA

Auto Regressive Integrated Moving Average.

ARIMA(p, d, q)

Where:

p = AR order
d = differencing order
q = MA order

Use ARIMA for non-stationary time series after differencing.

33 29. Reinforcement Learning

Reinforcement Learning learns through reward and punishment.

Important terms:

Term	Meaning
Agent	Learner/decision maker
Environment	World where agent acts
State	Current situation
Action	Move taken by agent
Reward	Feedback after action
Policy	Strategy used by agent
Value	Expected future reward
Q-value	Value of taking action in state

33.1 29.1 Markov Decision Process

MDP is a mathematical framework for reinforcement learning.

It contains:

- States
- Actions
- Transition probabilities
- Rewards
- Discount factor

33.2 29.2 Q-Learning

Q-learning learns the best action for each state.

Simple idea:

Try action -> get reward -> update Q-table -> improve next decision

34 30. Deep Learning and Neural Networks

Deep Learning uses artificial neural networks with many layers.

34.1 30.1 Neural Network Structure

Basic structure:

Input Layer -> Hidden Layers -> Output Layer

Each neuron does:

Weighted sum + bias -> activation function

34.2 30.2 Forward Propagation

Forward propagation means data moves from input to output.

Input -> hidden layers -> output prediction

34.3 30.3 Loss Function

Loss function measures how wrong the prediction is.

Examples:

MSE -> regression

Cross entropy -> classification

34.4 30.4 Backpropagation

Backpropagation updates weights by moving backward from error.

Prediction error -> gradients -> update weights

34.5 30.5 Activation Functions

Important activation functions:

- Sigmoid
- Tanh
- ReLU
- Softmax

34.5.1 Sigmoid

Output between 0 and 1. Used in binary classification output layer.

34.5.2 Tanh

Output between -1 and 1.

34.5.3 ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Most commonly used in hidden layers.

34.5.4 Softmax

Used for multi-class classification output layer.

35 31. CNN - Convolutional Neural Network

CNN is mainly used for image data.

Flow:

Image -> Convolution -> ReLU -> Pooling -> Flatten -> Fully Connected -> Output

35.1 31.1 Convolution Layer

A filter moves over the image and detects patterns.

Examples:

- Edges
- Corners
- Shapes
- Textures

35.2 31.2 ReLU Layer

ReLU removes negative values and keeps positive features.

35.3 31.3 Pooling Layer

Pooling reduces image size while keeping important features.

Types:

- Max pooling
- Average pooling

35.4 31.4 Fully Connected Layer

Final classification layer.

Example:

Cat / Dog / Car

36 32. RNN - Recurrent Neural Network

RNN is used for sequence data.

Examples:

- Text
- Speech
- Time series
- Language translation
- Stock sequences

RNN has memory of previous inputs.

Flow:

Current input + previous hidden state -> current output

Problem:

Vanishing gradient

Advanced models:

- LSTM
- GRU

37 33. Deep Learning Tools

Common deep learning tools:

Tool	Meaning
TensorFlow	Deep learning framework
Keras	High-level neural network API
PyTorch	Flexible deep learning framework
Theano	Older numerical computation library

38 34. Information Theory for ML - CDAC Add-on

Information theory helps ML measure uncertainty and information.

Important terms:

- Entropy
- Information Gain
- Gini Impurity
- Cross Entropy
- KL Divergence
- Mutual Information

38.1 34.1 Entropy

Entropy measures uncertainty.

High entropy -> mixed data

Low entropy -> pure data

Used in decision trees.

38.2 34.2 Information Gain

Information Gain measures how much uncertainty is reduced.

Information Gain = Entropy(parent) - Entropy(children)

Used to select the best split in decision trees.

38.3 34.3 Cross Entropy

Cross entropy is used as a loss function in classification.

Lower cross entropy -> better model

38.4 34.4 KL Divergence

KL Divergence measures difference between two probability distributions.

38.5 34.5 Mutual Information

Mutual Information measures dependency between two variables.

Use:

Feature selection

39 35. PAC Learning - CDAC Add-on

PAC means Probably Approximately Correct.

PAC learning asks:

Can a model learn a good function from limited training data?

PAC says the model should be:

Probably correct -> high probability

Approximately correct -> small error

Important terms:

- Hypothesis
- Concept class
- Error
- Confidence
- Sample complexity

Simple memory:

With enough data, a model can probably learn a nearly correct pattern.

40 36. VC Dimension - CDAC Add-on

VC Dimension measures model capacity.

Simple meaning:

How complex patterns can a model learn?

High VC dimension:

- Can learn complex patterns
- Higher overfitting risk
- Needs more data

Low VC dimension:

- Simple model
- May underfit
- Needs less data

Example:

Linear model -> lower VC dimension
Deep neural network -> high VC dimension

41 37. Structural Risk Minimization - CDAC Add-on

Structural Risk Minimization, or SRM, chooses a model by balancing training error and model complexity.

Goal:

Low training error + low complexity = better generalization

Difference:

ERM -> minimizes training error
SRM -> minimizes training error + complexity

42 38. Occam's Razor - CDAC Add-on

Occam's Razor says:

Prefer the simplest model that performs well.

ML meaning:

Do not use a complex model if a simple model gives similar performance.

Example:

If Logistic Regression and XGBoost both give 85% accuracy, prefer Logistic Regression
↪ for simplicity and interpretability.

43 39. No Free Lunch Theorem - CDAC Add-on

No Free Lunch theorem says:

No single ML algorithm is best for every problem.

Meaning:

- Linear Regression may work for linear data.
- Naive Bayes may work for text.
- Random Forest may work for tabular data.
- SVM may work for smaller high-dimensional data.

So always try multiple models and compare.

44 40. Model Stability - CDAC Add-on

Model stability means model performance should not change drastically when training data changes slightly.

Unstable model:

Small data change -> very different predictions

Stable model:

Small data change -> similar predictions

Ways to improve stability:

- Cross-validation
- Regularization
- More data
- Feature selection
- Ensemble methods

45 41. Complete Scikit-Learn Workflow

This is very important for lab and projects.

45.1 41.1 Standard ML Workflow

1. Load dataset
2. Check head, tail, shape, info, describe
3. Check missing values
4. Clean data
5. Separate X and y
6. Encode categorical columns
7. Scale numerical columns if needed
8. Train-test split
9. Train model
10. Predict
11. Evaluate
12. Tune hyperparameters
13. Save model
14. Deploy model

45.2 41.2 Complete Classification Pipeline Template

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

X = df[["Age", "Salary", "Experience"]]
y = df["Purchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=3),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC(kernel="linear")
}

for name, model in models.items():
    pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("model", model)
    ])

    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)

    print("Model:", name)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print("Classification Report:")
    print(classification_report(y_test, y_pred))
    print("-" * 50)

```

45.3 41.3 Important Mistakes to Avoid

Wrong train_test_split order:
X_train, y_train, X_test, y_test

Correct:
X_train, X_test, y_train, y_test

Wrong SVM prediction:
y_pred = model(X_test)

Correct:
y_pred = model.predict(X_test)

Wrong loop variable:
for name, models in models.items():

```
Correct:
for name, model in models.items():
```

```
Wrong spelling:
y_tets
```

```
Correct:
y_test
```

46 42. Lab Exam Must-Know Programs

Practice these programs in sequence:

1. Load dataset using pandas.
2. Print first 5 and last 5 rows.
3. Check shape, info, describe.
4. Check missing values.
5. Check duplicate rows.
6. Check correlation.
7. Encode categorical variables.
8. Apply StandardScaler.
9. Perform train-test split.
10. Train Linear Regression.
11. Train Logistic Regression.
12. Train Decision Tree.
13. Train Random Forest.
14. Train KNN.
15. Train Naive Bayes.
16. Train SVM.
17. Evaluate using accuracy.
18. Print confusion matrix.
19. Print classification report.
20. Apply K-Means clustering.
21. Use elbow method.
22. Calculate silhouette score.
23. Apply PCA.
24. Use cross-validation.
25. Use GridSearchCV.
26. Build complete ML pipeline.
27. Save model using joblib.
28. Load model and make prediction.

47 43. Which Algorithm to Use When?

Problem Type	Good Starting Algorithms
House price / salary prediction	Linear Regression, Ridge, Lasso, Random Forest Regressor, XGBoost
Binary classification	Logistic Regression, Decision Tree, Random Forest, SVM
Text classification	Naive Bayes, Logistic Regression
Customer segmentation	K-Means, Hierarchical Clustering, DBSCAN
Dimensionality reduction	PCA, Kernel PCA, Random Projection
Market basket analysis	Apriori, Association Rules

Problem Type	Good Starting Algorithms
Recommendation	Content-based, Collaborative, Hybrid
Time series	AR, MA, ARMA, ARIMA, Exponential Smoothing
Image classification	CNN
Sequence/text/time data	RNN, LSTM, GRU
Complex tabular ML	Random Forest, XGBoost, LightGBM, CatBoost

48 44. One-Line Revision

Regression predicts numbers.
 Classification predicts classes.
 Clustering finds groups.
 PCA reduces features.
 Association rules find item relationships.
 Recommendation systems suggest items.
 Anomaly detection finds unusual cases.
 Time series predicts future values using time order.
 Reinforcement learning learns through reward.
 Deep learning uses neural networks.
 CNN is for images.
 RNN is for sequences.
 Regularization reduces overfitting.
 Cross-validation checks generalization.
 GridSearchCV finds best hyperparameters.
 No Free Lunch means no algorithm is best for all problems.
 Occam's Razor means prefer the simplest good model.
 VC dimension measures model capacity.
 PAC learning asks whether a model can probably learn approximately correct rules.

49 45. Final Study Order for CDAC ML

Use this order for revision:

1. AI vs ML vs DL
2. ML life cycle
3. Types of ML
4. Data preprocessing and EDA
5. Supervised learning
6. Regression
7. Regression metrics
8. Overfitting, underfitting, bias-variance
9. Regularization
10. Cross-validation
11. Classification
12. Confusion matrix and metrics
13. Logistic Regression
14. SVM
15. Decision Tree
16. Random Forest
17. KNN
18. Ensemble learning
19. GBM, XGBoost, LightGBM, CatBoost
20. Naive Bayes
21. LDA and QDA

22. Unsupervised learning
23. K-Means
24. Hierarchical clustering
25. DBSCAN and Fuzzy C-Means
26. Cluster evaluation
27. PCA
28. Kernel PCA and Random Projection
29. Association Rule Mining
30. Recommendation Systems
31. Anomaly Detection
32. Time Series Forecasting
33. Reinforcement Learning
34. Deep Learning basics
35. CNN
36. RNN
37. PAC Learning
38. VC Dimension
39. SRM, Occam's Razor, No Free Lunch theorem
40. Complete scikit-learn pipeline practice

50 46. Final Checklist

Before your ML lab exam or interview, you should be able to explain and code:

- What is ML and how it differs from AI and DL.
- ML life cycle from problem statement to deployment.
- Difference between supervised, unsupervised, and reinforcement learning.
- Difference between regression and classification.
- Linear Regression, Logistic Regression, Decision Tree, Random Forest, KNN, Naive Bayes, SVM.
- Regression metrics: MAE, MSE, RMSE, R2.
- Classification metrics: accuracy, precision, recall, F1, ROC-AUC.
- Overfitting, underfitting, bias, variance.
- Regularization: Ridge, Lasso, Elastic Net.
- Cross-validation and GridSearchCV.
- K-Means, Hierarchical Clustering, DBSCAN.
- PCA and dimensionality reduction.
- Association rules: support, confidence, lift.
- Recommendation systems.
- Time series basics: trend, seasonality, ARIMA.
- Deep learning basics: neural network, activation, backpropagation, CNN, RNN.
- CDAC theory: information theory, PAC learning, VC dimension, SRM, Occam's Razor, No Free Lunch theorem.

51 Part B - Expanded 100 Percent CDAC Syllabus Coverage

52 6. Information Theory for Machine Learning

Information theory explains uncertainty, impurity, and information content. It is important in decision trees, feature selection, classification loss functions, and model comparison.

52.1 6.1 Entropy

Entropy measures uncertainty or disorder in a set of labels.

Formula:

$$\text{Entropy}(S) = - \sum (p_i * \log_2(p_i))$$

Where p_i is the probability of class i .

Interpretation:

- Entropy = 0 means the node is pure. Example: all records are Yes.
- Entropy is high when classes are mixed. Example: half Yes, half No.
- Decision trees try to reduce entropy after splitting.

Example:

Dataset has 10 rows:

6 Yes, 4 No

$$p(\text{Yes}) = 6/10 = 0.6$$

$$p(\text{No}) = 4/10 = 0.4$$

$$\text{Entropy} = -(0.6 \log_2 0.6 + 0.4 \log_2 0.4)$$

52.2 6.2 Information Gain

Information Gain tells how much uncertainty is reduced after a split.

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

A decision tree chooses the feature that gives the highest information gain.

Simple memory:

Best split = maximum reduction in impurity.

52.3 6.3 Cross Entropy

Cross entropy is commonly used as a classification loss function.

For binary classification:

$$\text{Loss} = -[y \log(p) + (1-y) \log(1-p)]$$

Where:

- y is actual class, either 0 or 1.
- p is predicted probability for class 1.

If the model is confident and correct, loss is low. If the model is confident and wrong, loss is very high.

52.4 6.4 KL Divergence

KL Divergence measures how different one probability distribution is from another.

$$\text{KL}(P || Q) = \sum P(x) \log(P(x) / Q(x))$$

In ML, it is used to compare actual distribution and predicted distribution. It is also used in probabilistic models, variational autoencoders, and information-based learning.

52.5 6.5 Mutual Information

Mutual Information measures how much knowing one variable tells us about another variable.

Example:

If knowing Education tells a lot about Salary, mutual information is high.
If knowing Customer ID tells nothing useful about Purchase, mutual information is low.

Use in ML:

- Feature selection
- Finding dependency between input and target
- Understanding non-linear relationships

52.6 6.6 Gini Impurity

Gini impurity is another impurity measure used in CART decision trees.

$$\text{Gini} = 1 - \sum(p_i^2)$$

Low Gini means pure node. High Gini means mixed node.

Comparison:

Entropy uses log.
Gini uses squared probabilities.
Both are used to select good splits.
Gini is often faster to compute.

53 7. Learning Theory: PAC, VC Dimension, SRM, Occam, No Free Lunch

These topics explain why models learn, when they generalize, and why no model is always best.

53.1 7.1 Generalization

Generalization means a model performs well on unseen data, not only training data.

Good model:

Training score: high
Testing score: high

Bad model due to overfitting:

Training score: very high
Testing score: low

Bad model due to underfitting:

Training score: low
Testing score: low

53.2 7.2 PAC Learning

PAC means Probably Approximately Correct.

The main question in PAC learning is:

Can a model learn a nearly correct rule with high probability using finite data?

Important terms:

- Concept: the true rule we want to learn.
- Hypothesis: the rule learned by the model.
- Hypothesis class: all possible rules the model can learn.
- Error: difference between hypothesis and true concept.
- Confidence: how sure we are that learned rule is good.
- Sample complexity: how many training examples are needed.

Simple meaning:

With enough examples, a suitable model can probably learn an approximately correct
↪ pattern.

53.3 7.3 VC Dimension

VC dimension measures capacity of a model.

Simple meaning:

VC dimension = how complex a pattern the model can separate.

High VC dimension:

- Can learn complex patterns.
- Needs more data.
- Higher overfitting risk.

Low VC dimension:

- Simpler model.
- Needs less data.
- May underfit complex data.

Example:

Linear model: lower capacity.
 Decision tree with unlimited depth: high capacity.
 Deep neural network: very high capacity.

53.4 7.4 Empirical Risk Minimization

Empirical Risk Minimization, or ERM, means choosing the model that minimizes training error.

Problem:

Only minimizing training error can create overfitting.

53.5 7.5 Structural Risk Minimization

Structural Risk Minimization, or SRM, balances training error and model complexity.

$SRM = \text{training error} + \text{complexity penalty}$

This is the principle behind regularization.

Examples:

- Ridge Regression penalizes large coefficients.
- Lasso Regression penalizes absolute coefficients.
- Decision Tree pruning reduces complexity.
- Early stopping stops neural network training before overfitting.

53.6 7.6 Occam's Razor

Occam's Razor says:

Prefer the simplest model that explains the data well.

Example:

If Logistic Regression gives 86% accuracy and XGBoost gives 87%, Logistic Regression
 ↪ may be better for interpretability.

Interview answer:

We should not blindly choose the most complex model. We should choose the simplest
 ↪ model that generalizes well.

53.7 7.7 No Free Lunch Theorem

No Free Lunch theorem says:

No single algorithm is best for all problems.

Meaning:

- Linear Regression works well for linear continuous data.
- Naive Bayes often works well for text classification.
- Random Forest works well on many tabular datasets.
- SVM works well on smaller high-dimensional datasets.

- CNN works well for images.
- RNN/LSTM works well for sequence data.

Practical rule:

Try multiple models, evaluate fairly, then select the best one based on metric and
↪ business need.

53.8 7.8 Model Stability

A stable model gives similar results even if the training data changes slightly.

Ways to improve stability:

- Use cross-validation.
- Use regularization.
- Use more data.
- Remove noisy features.
- Use ensembles like Random Forest.
- Avoid over-complex models on small data.

54 8. Data Preparation, Feature Engineering, and Pipelines

Data preparation is usually the most time-consuming part of ML. A strong ML model depends on clean, meaningful, and correctly transformed data.

54.1 8.1 Basic Data Inspection

Always start with:

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())
print(df.tail())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

54.2 8.2 Missing Values

Common methods:

- Drop rows if missing values are very few.
- Fill numeric values with mean or median.
- Fill categorical values with mode.
- Use domain-specific value.
- Use model-based imputation for advanced cases.

Example:

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
```

54.3 8.3 Outliers

Outliers are extreme values.

Detection methods:

- Boxplot
- IQR method
- Z-score
- Domain rules

IQR method:

```
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df["Salary"] < lower) | (df["Salary"] > upper)]
```

54.4 8.4 Encoding Categorical Variables

Label Encoding:

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
```

One-Hot Encoding:

```
df = pd.get_dummies(df, columns=["City"], drop_first=True)
```

Rule:

- Use label encoding for ordinal categories.
- Use one-hot encoding for nominal categories.

54.5 8.5 Feature Scaling

Scaling changes feature range.

Important for:

- KNN
- SVM
- Logistic Regression
- Linear Regression with regularization
- K-Means
- PCA
- Neural Networks

Not very important for tree-based models:

- Decision Tree
- Random Forest
- XGBoost
- LightGBM
- CatBoost

StandardScaler:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

MinMaxScaler:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

RobustScaler is useful when outliers exist.

54.6 8.6 Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Important memory:

Correct order:

```
X_train, X_test, y_train, y_test
```

54.7 8.7 Pipelines

A pipeline combines preprocessing and model training.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

Benefits:

- Cleaner code.
- Avoids data leakage.
- Easier cross-validation.
- Easier deployment.

54.8 8.8 Data Leakage

Data leakage happens when information from test data enters training.

Wrong:

```
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y)
```

Correct:

```
X_train, X_test, y_train, y_test = train_test_split(X, y)
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Pipeline handles this automatically.

55 9. Cross-Validation and Hyperparameter Tuning

55.1 9.1 Why Cross-Validation?

A single train-test split may be lucky or unlucky. Cross-validation gives a more reliable estimate.

55.2 9.2 K-Fold Cross-Validation

Steps:

1. Split data into K equal folds.
2. Train on K-1 folds.
3. Test on remaining fold.
4. Repeat K times.
5. Average the score.

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
print(scores)
print(scores.mean())
```

55.3 9.3 Stratified K-Fold

Used for classification. It keeps class distribution similar in every fold.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skf, scoring="f1")
```

55.4 9.4 Time Series Split

For time series, never shuffle future and past randomly.

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
for train_idx, test_idx in tscv.split(X):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
```

55.5 9.5 GridSearchCV

Tries all combinations of hyperparameters.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

params = {
    "n_estimators": [100, 200],
    "max_depth": [None, 5, 10],
    "min_samples_split": [2, 5]
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
```

```
    params,  
    cv=5,  
    scoring="f1",  
    n_jobs=-1  
)  
grid.fit(X_train, y_train)  
print(grid.best_params_)  
print(grid.best_score_)
```

55.6 9.6 RandomizedSearchCV

Tries random combinations. Useful when search space is large.

```
from sklearn.model_selection import RandomizedSearchCV  
  
random_search = RandomizedSearchCV(  
    estimator=model,  
    param_distributions=params,  
    n_iter=20,  
    cv=5,  
    random_state=42  
)
```


56 10. Clustering, WSS, Best K, Distance Measures, PCA, Random Projection

This chapter covers the mall customers lab and the self-learning points from the syllabus.

56.1 10.1 What is Clustering?

Clustering is an unsupervised learning method where the model groups similar data points without using labels.

Example:

Mall customers can be grouped based on annual income and spending score.

56.2 10.2 Why Clustering is Subjective and Exploratory

Clustering is subjective because there is no single correct answer.

Reasons:

- No target label is available.
- Different features create different clusters.
- Different scaling methods create different clusters.
- Different distance measures create different clusters.
- Different values of K create different business interpretations.

Example:

Using Age + Income may create age-income groups.
Using Income + Spending Score may create customer value groups.
Both can be valid depending on business objective.

Therefore clustering is exploratory. We do not ask only “Is accuracy high?” Instead we ask:

- Are clusters separated?
- Are clusters meaningful?
- Can business take action from clusters?
- Are clusters stable?

56.3 10.3 K-Means Clustering

K-Means partitions data into K clusters.

Steps:

1. Choose K.
2. Randomly initialize K centroids.
3. Assign each point to nearest centroid.
4. Recalculate centroids.
5. Repeat until centroids stop changing.

56.4 10.4 WSS / Inertia

WSS means Within-Cluster Sum of Squares. In scikit-learn it is called `inertia_`.

WSS = sum of squared distances from each point to its cluster centroid

Low WSS means points are close to their centroids.

But WSS always decreases when K increases. Therefore we use the elbow method.

56.5 10.5 Elbow Method for Best K

Train K-Means for multiple K values and plot WSS.

Choose the K where WSS reduction slows down sharply.

This point looks like an elbow in the plot.

56.6 10.6 Mall Customers Lab

Dataset columns usually include:

- CustomerID
- Gender
- Age
- Annual Income
- Spending Score

Typical features for clustering:

Annual Income and Spending Score

Full lab code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# 1. Load dataset
df = pd.read_csv("mall_customers.csv")
print(df.head())
print(df.info())
print(df.isnull().sum())

# 2. Select features
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

# 3. Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Calculate WSS for different K
wss = []
sil_scores = []
K_range = range(2, 11)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    wss.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, km.labels_))

# 5. Plot elbow curve
plt.plot(K_range, wss, marker="o")
plt.xlabel("Number of clusters K")
plt.ylabel("WSS / Inertia")
plt.title("Elbow Method")
plt.show()
```

```
# 6. Plot silhouette score
plt.plot(K_range, sil_scores, marker="o")
plt.xlabel("Number of clusters K")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score for K")
plt.show()

# 7. Final model
best_k = 5
final_model = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df["Cluster"] = final_model.fit_predict(X_scaled)

print(df.groupby("Cluster")[["Annual Income (k$)", "Spending Score (1-100)"]].mean())
```

56.7 10.7 How to Form Number of Clusters by Observation

Use scatter plot:

```
plt.scatter(df["Annual Income (k$)"], df["Spending Score (1-100)"])
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Distribution")
plt.show()
```

Observe natural groups visually. Then confirm with elbow method and silhouette score.

56.8 10.8 Cluster Profiling

After clustering, explain each group.

Example for mall customers:

```
Cluster 0: High income, high spending - target premium offers.
Cluster 1: High income, low spending - encourage with personalized offers.
Cluster 2: Low income, high spending - price-sensitive loyal buyers.
Cluster 3: Low income, low spending - low priority segment.
Cluster 4: Medium income, medium spending - general customers.
```

56.9 10.9 Distance Measures

Distance measures decide similarity.

Euclidean distance:

Straight-line distance. Common in K-Means.

Manhattan distance:

City-block distance. Useful when movement is grid-like.

Cosine similarity:

Measures angle between vectors. Common in text and recommendation systems.

Minkowski distance:

Generalized distance. Euclidean and Manhattan are special cases.

56.10 10.10 Impact of Scaling on Clustering

K-Means uses distance. Large-scale features dominate.

Example:

Age range: 18 to 70
Income range: 15000 to 200000

Income will dominate distance if not scaled.

Therefore always scale features for K-Means, KNN, SVM, PCA, and neural networks.

56.11 10.11 Clustering Evaluation vs Supervised Evaluation

Supervised learning has actual labels. We can calculate accuracy, F1, MAE, RMSE.

Clustering usually has no labels. Therefore evaluation is different.

Internal metrics:

- WSS / Inertia
- Silhouette Score
- Davies-Bouldin Index
- Calinski-Harabasz Score

Business evaluation:

- Are clusters explainable?
- Are clusters useful for marketing?
- Are clusters stable across samples?

56.12 10.12 PCA as Representation, Not Only Compression

PCA creates new axes that represent directions of maximum variance.

PCA is not just compression. It is representation learning because it transforms data into a new coordinate system.

Uses:

- Visualization
- Noise reduction
- Removing multicollinearity
- Faster training
- Understanding major variation in data

56.13 10.13 Random Projection as Representation

Random Projection reduces dimensions using random matrices while approximately preserving distances.

Use when:

- Data is very high-dimensional.
- PCA is too slow.
- Approximation is acceptable.

Comparison:

PCA: data-driven, accurate, slower.

Random Projection: random transformation, faster, approximate.

57 11. Evaluation Metrics: Classification, Regression, Ranking, and LogLoss

57.1 11.1 Confusion Matrix

Confusion matrix contains:

TP = True Positive
TN = True Negative
FP = False Positive
FN = False Negative

57.2 11.2 Accuracy

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Use when classes are balanced.

Problem:

If 95% emails are not spam, a model predicting all not-spam gets 95% accuracy but is
↪ useless for spam detection.

57.3 11.3 Precision

$$\text{Precision} = TP / (TP + FP)$$

Out of predicted positives, how many are actually positive?

Use when false positives are costly.

Example:

Spam detection: do not mark important email as spam.

57.4 11.4 Recall / Sensitivity

$$\text{Recall} = TP / (TP + FN)$$

Out of actual positives, how many did we catch?

Use when false negatives are costly.

Example:

Cancer detection, fraud detection.

57.5 11.5 F1-Score

F1-score balances precision and recall.

$$F1 = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$$

Use when dataset is imbalanced.

57.6 11.6 Specificity

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

It measures how well the model detects negative cases.

57.7 11.7 ROC Curve and ROC-AUC

ROC curve plots:

True Positive Rate vs False Positive Rate

Where:

$$\text{TPR} = \text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

AUC interpretation:

- 1.0 = perfect classifier.
- 0.5 = random classifier.
- Below 0.5 = worse than random.

Python:

```
from sklearn.metrics import roc_auc_score

y_prob = model.predict_proba(X_test)[: , 1]
auc = roc_auc_score(y_test, y_prob)
print(auc)
```

57.8 11.8 LogLoss

LogLoss measures quality of predicted probabilities.

Binary log loss:

$$\text{LogLoss} = - \text{average}[y \log(p) + (1-y) \log(1-p)]$$

Important:

- Low LogLoss is good.
- Confident wrong predictions are heavily punished.
- Useful when probability quality matters.

Python:

```
from sklearn.metrics import log_loss

y_prob = model.predict_proba(X_test)
print(log_loss(y_test, y_prob))
```

57.9 11.9 Regression Metrics

57.9.1 MAE

MAE = average absolute error

Easy to understand and robust compared to MSE.

57.9.2 MSE

MSE = average squared error

Penalizes large errors strongly.

57.9.3 RMSE

$RMSE = \sqrt{MSE}$

Same unit as target.

57.9.4 R2 Score

$R^2 = 1 - RSS/TSS$

Interpretation:

- R^2 close to 1 is good.
- R^2 close to 0 means weak model.
- R^2 can be negative if model is worse than predicting mean.

57.9.5 Adjusted R2

Adjusted R^2 penalizes unnecessary features.

Use when comparing multiple regression models with different number of features.

57.10 11.10 Metric Selection Guide

Use this guide:

Balanced classification: Accuracy

Imbalanced classification: F1-score, Precision, Recall, ROC-AUC

Probability quality: LogLoss

Regression with understandable error: MAE

Regression with large-error penalty: RMSE

Regression goodness-of-fit: R^2 , Adjusted R^2

Recommendation ranking: Recall@K, Precision@K, MRR, MAP@K, NDCG

58 12. Decision Trees and CART

58.1 12.1 What is a Decision Tree?

Decision Tree is a supervised learning algorithm used for classification and regression.

It uses tree-like rules:

Root Node -> Decision Node -> Leaf Node

Example:

If Income > 50000 and Credit_History = 1 -> Loan Approved
Else -> Loan Rejected

58.2 12.2 CART

CART means Classification and Regression Tree.

- Classification tree predicts class labels.
- Regression tree predicts continuous values.

58.3 12.3 Important Terms

Root node:

First split of the tree.

Decision node:

Internal node where condition is checked.

Leaf node:

Final output node.

Splitting:

Dividing data based on a feature condition.

Pruning:

Removing unnecessary branches to reduce overfitting.

58.4 12.4 Splitting Criteria

Classification:

- Gini impurity
- Entropy and Information Gain

Regression:

- Mean Squared Error
- Mean Absolute Error

58.5 12.5 Decision Tree Hyperparameters

Important hyperparameters:


```
criterion
max_depth
min_samples_split
min_samples_leaf
max_features
class_weight
```

Meaning:

- max_depth: maximum depth of tree.
- min_samples_split: minimum samples required to split a node.
- min_samples_leaf: minimum samples in leaf node.
- max_features: number of features considered at each split.
- class_weight: handles class imbalance.

58.6 12.6 Overfitting in Decision Trees

Decision trees easily overfit if allowed to grow fully.

Control methods:

- Limit max_depth.
- Increase min_samples_leaf.
- Use pruning.
- Use cross-validation.
- Use Random Forest.

58.7 12.7 Decision Tree Code

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

59 13. Random Forest, GBM, Stacking, XGBoost, LightGBM, CatBoost

59.1 13.1 Ensemble Learning

Ensemble learning combines multiple models to improve performance.

Main types:

- Voting
- Bagging
- Boosting
- Stacking

59.2 13.2 Voting

Multiple models vote for the final answer.

Hard voting:

```
Final class = majority vote
```

Soft voting:

```
Final class = average predicted probability
```

59.3 13.3 Bagging

Bagging means Bootstrap Aggregating.

Steps:

1. Create multiple random samples from training data.
2. Train one model on each sample.
3. Combine predictions.

Random Forest is the most common bagging algorithm.

59.4 13.4 Random Forest

Random Forest trains many decision trees using random samples and random feature subsets.

For classification:

```
Final prediction = majority vote
```

For regression:

```
Final prediction = average of trees
```

Advantages:

- Reduces overfitting compared to one decision tree.
- Handles non-linear data.
- Works well on tabular data.
- Gives feature importance.

Disadvantages:

- Less interpretable than single tree.
- Larger model size.
- Slower than simple models.

Code:

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
```

59.5 13.5 Boosting

Boosting trains weak learners sequentially. Each new learner focuses on mistakes of previous learners.

Simple idea:

```
Model 1 makes errors.
Model 2 focuses on Model 1 errors.
Model 3 focuses on remaining errors.
Final model combines all learners.
```

59.6 13.6 Gradient Boosting Machine

GBM uses gradient descent to reduce loss by adding weak learners.

Three main components:

- Loss function
- Weak learner, usually shallow decision tree
- Additive model

Code:

```
from sklearn.ensemble import GradientBoostingClassifier

gbm = GradientBoostingClassifier(random_state=42)
gbm.fit(X_train, y_train)
y_pred = gbm.predict(X_test)
```

59.7 13.7 XGBoost

XGBoost means Extreme Gradient Boosting.

Important features:

- Regularization to reduce overfitting.
- Handles missing values.
- Parallel processing.
- Tree pruning.
- Good performance on structured/tabular data.

Common hyperparameters:

```
n_estimators
learning_rate
max_depth
subsample
colsample_bytree
```

```
reg_alpha  
reg_lambda
```

Code:

```
from xgboost import XGBClassifier  
  
xgb = XGBClassifier(  
    n_estimators=200,  
    learning_rate=0.05,  
    max_depth=4,  
    random_state=42,  
    eval_metric="logloss"  
)  
xgb.fit(X_train, y_train)  
y_pred = xgb.predict(X_test)
```

59.8 13.8 LightGBM

LightGBM is a fast gradient boosting framework.

Important idea:

LightGBM grows trees leaf-wise rather than level-wise.

Advantages:

- Fast training.
- Good for large datasets.
- Handles high-dimensional data.

Risk:

- Can overfit on small datasets if not tuned.

Common hyperparameters:

```
num_leaves  
learning_rate  
n_estimators  
max_depth  
feature_fraction  
bagging_fraction
```

59.9 13.9 CatBoost

CatBoost is gradient boosting designed to handle categorical features effectively.

Advantages:

- Handles categorical variables well.
- Less preprocessing needed.
- Strong performance with default settings.
- Reduces target leakage using ordered boosting.

Use when:

Dataset has many categorical columns.

59.10 13.10 Model Stacking

Stacking combines predictions from multiple base models using a meta-model.

Flow:

Base Models -> Predictions -> Meta Model -> Final Prediction

Example:

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

estimators = [
    ("rf", RandomForestClassifier(random_state=42)),
    ("svm", SVC(probability=True, random_state=42))
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(max_iter=1000)
)
stack.fit(X_train, y_train)
```

59.11 13.11 Bagging vs Boosting vs Stacking

Bagging: parallel models, reduces variance, example Random Forest.

Boosting: sequential models, reduces bias, example XGBoost.

Stacking: different models combined by meta-model.

60 14. Session 6 and 7 Lab: Random Forest, SVM, Logistic Regression

Goal:

Implement Random Forest, SVM, and Logistic Regression classification algorithms. Check classification report and F1-score for all three.

Full generic code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, f1_score, classification_report,
    ↪ confusion_matrix, roc_auc_score, log_loss

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# 1. Load dataset
df = pd.read_csv("classification_dataset.csv")

# 2. Basic checks
print(df.head())
print(df.info())
print(df.isnull().sum())

# 3. Separate X and y
X = df.drop("target", axis=1)
y = df["target"]

# 4. One-hot encode categorical columns if any
X = pd.get_dummies(X, drop_first=True)

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 6. Define models
models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=1000))
    ]),
    "SVM": Pipeline([
        ("scaler", StandardScaler()),
        ("model", SVC(kernel="rbf", probability=True, random_state=42))
    ]),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42)
}

# 7. Train and evaluate
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

```

print("\n=====")
print(name)
print("=====")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred, average="weighted"))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred))

if hasattr(model, "predict_proba"):
    y_prob = model.predict_proba(X_test)
    if len(set(y)) == 2:
        print("ROC-AUC:", roc_auc_score(y_test, y_prob[:, 1]))
        print("LogLoss:", log_loss(y_test, y_prob))

```

Conclusion format for lab file:

Logistic Regression is simple and interpretable. It works well when classes are
 ↳ linearly separable.

SVM performs well with scaled data and can capture non-linear boundaries using kernels.
 Random Forest usually performs strongly on tabular data and handles non-linearity, but
 ↳ is less interpretable.

The final model should be selected based on F1-score, ROC-AUC, LogLoss, and business
 ↳ requirement.

61 15. Bayesian Analysis and Naive Bayes

61.1 15.1 Bayesian Thinking

Bayesian analysis updates belief when new evidence is observed.

Bayes theorem:

$$P(H \mid E) = P(E \mid H) * P(H) / P(E)$$

Where:

- $P(H \mid E)$ = posterior probability.
- $P(E \mid H)$ = likelihood.
- $P(H)$ = prior probability.
- $P(E)$ = evidence.

Simple example:

H = email is spam

E = email contains word "offer"

$P(\text{spam} \mid \text{offer})$ = probability email is spam given it contains offer

61.2 15.2 Naive Bayes Classifier

Naive Bayes applies Bayes theorem with a naive assumption:

All features are independent given the class.

Even though this assumption is often not fully true, Naive Bayes works well in many problems.

61.3 15.3 Probability Calculation Example

Suppose we classify weather as Play or No Play.

$$P(\text{Play} \mid \text{Sunny}) = P(\text{Sunny} \mid \text{Play}) * P(\text{Play}) / P(\text{Sunny})$$

If:

$$P(\text{Sunny} \mid \text{Play}) = 0.3$$

$$P(\text{Play}) = 0.6$$

$$P(\text{Sunny}) = 0.4$$

Then:

$$P(\text{Play} \mid \text{Sunny}) = 0.3 * 0.6 / 0.4 = 0.45$$

The class with highest posterior probability is selected.

61.4 15.4 Types of Naive Bayes

Gaussian Naive Bayes:

Used for continuous numeric features. Assumes normal distribution.

Multinomial Naive Bayes:

Used for count data. Common in text classification using word counts.

Bernoulli Naive Bayes:

Used for binary features: word present or absent.

61.5 15.5 Naive Bayes Code

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

61.6 15.6 Text Classification with MultinomialNB

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("model", MultinomialNB())
])

pipe.fit(X_train_text, y_train)
y_pred = pipe.predict(X_test_text)
```

61.7 15.7 Pros and Cons

Pros:

- Simple and fast.
- Works well for text classification.
- Works with small data.
- Handles high-dimensional sparse data.

Cons:

- Independence assumption is often unrealistic.
- Probability estimates may be poor.
- Struggles when features strongly depend on each other.

62 16. Linear Discriminant Analysis and QDA

62.1 16.1 What is LDA?

Linear Discriminant Analysis, or LDA, is a supervised classification method. It finds a linear combination of features that separates classes.

LDA can also be used for dimensionality reduction.

62.2 16.2 LDA Goal

LDA tries to:

Maximize separation between classes
Minimize spread within each class

62.3 16.3 LDA Assumptions

- Features are numeric.
- Features are normally distributed within each class.
- Classes have similar covariance matrices.
- Observations are independent.

62.4 16.4 LDA vs PCA

PCA is unsupervised and maximizes variance.
LDA is supervised and maximizes class separation.

62.5 16.5 LDA Code

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import classification_report

lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
y_pred = lda.predict(X_test)
print(classification_report(y_test, y_pred))
```

62.6 16.6 QDA

Quadratic Discriminant Analysis is similar to LDA but allows each class to have a different covariance matrix.

Difference:

LDA: linear decision boundary.
QDA: curved/quadratic decision boundary.

Use QDA when class distributions have different shapes.

63 17. K-Nearest Neighbors Algorithm

63.1 17.1 What is KNN?

K-Nearest Neighbors is a supervised learning algorithm used for classification and regression.

It is called lazy learning because it does not build a strong training model. It stores the training data and predicts using nearest examples.

63.2 17.2 KNN Working

Steps:

1. Choose K.
2. Calculate distance from new point to all training points.
3. Select K nearest neighbors.
4. For classification, use majority vote.
5. For regression, use average value.

63.3 17.3 Choosing K

Small K:

More sensitive to noise. Can overfit.

Large K:

Smoother boundary. Can underfit.

Usually choose K using cross-validation.

63.4 17.4 Distance Measures

Euclidean distance:

```
sqrt(sum((x_i - y_i)^2))
```

Manhattan distance:

```
sum(|x_i - y_i|)
```

Minkowski distance:

General form of Euclidean and Manhattan.

63.5 17.5 Why Scaling is Important for KNN

KNN depends directly on distance. If one feature has a large scale, it dominates distance.

Therefore always scale data before KNN.

63.6 17.6 KNN Lab Code

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
```

```

from sklearn.metrics import accuracy_score, f1_score, classification_report,
    ↪ confusion_matrix

# Load data
df = pd.read_csv("dataset.csv")

# Separate features and target
X = df.drop("target", axis=1)
y = df["target"]

# Encode categorical variables
X = pd.get_dummies(X, drop_first=True)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Pipeline
knn_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])

knn_pipe.fit(X_train, y_train)
y_pred = knn_pipe.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred, average="weighted"))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

63.7 17.7 Find Best K

```

scores = []
for k in range(1, 31):
    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    scores.append((k, f1_score(y_test, y_pred, average="weighted")))

print(sorted(scores, key=lambda x: x[1], reverse=True)[:5])

```

63.8 17.8 KNN Pros and Cons

Pros:

- Simple.
- No training time.
- Works for classification and regression.

Cons:

- Slow prediction on large datasets.
- Sensitive to scaling.

- Sensitive to irrelevant features.
- Needs careful K selection.

64 18. Regression: Linear, Polynomial, Ridge, Lasso, Elastic-Net

64.1 18.1 Regression as Modeling, Not Only Curve Fitting

Regression should not be seen only as drawing a curve. It is a modeling tool to understand relationships and make predictions.

A good regression model should:

- Capture meaningful relationship.
- Generalize on unseen data.
- Have interpretable coefficients where possible.
- Avoid overfitting.
- Provide useful error metrics.

64.2 18.2 Simple Linear Regression

Used when one independent variable predicts one continuous target.

$$y = b_0 + b_1x$$

Example:

Experience -> Salary
Distance -> Delivery Time

64.3 18.3 Multiple Linear Regression

Used when multiple independent variables predict one continuous target.

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

64.4 18.4 Linear Regression Assumptions

- Linear relationship.
- No multicollinearity.
- Normal residuals.
- Homoscedasticity.
- No autocorrelation of errors.

64.5 18.5 Polynomial Regression

Used when relation is non-linear.

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$$

Risk:

High degree polynomial can overfit.

64.6 18.6 Regularization in High-Dimensional Data

High-dimensional data means many features. In such data, model can overfit and coefficients may become very large.

Regularization adds penalty to loss function.

Benefits:

- Reduces overfitting.
- Controls coefficient magnitude.
- Improves generalization.
- Handles multicollinearity.

64.7 18.7 Ridge Regression

Ridge uses L2 penalty.

```
Loss = MSE + alpha * sum(coef^2)
```

Effect:

- Shrinks coefficients.
- Does not usually make coefficients exactly zero.
- Good when most features are useful.

64.8 18.8 Lasso Regression

Lasso uses L1 penalty.

```
Loss = MSE + alpha * sum(|coef|)
```

Effect:

- Can shrink some coefficients to zero.
- Performs feature selection.
- Good when only some features are important.

64.9 18.9 ElasticNet Regression

ElasticNet combines L1 and L2 penalties.

Use when:

- Many correlated features exist.
- You want both feature selection and coefficient shrinkage.

64.10 18.10 Bias-Variance Comparison of Regression Techniques

Linear Regression:

Low complexity, can underfit non-linear data.

Polynomial Regression:

Higher complexity, can fit curves, may overfit if degree is high.

Ridge Regression:

Reduces variance by shrinking coefficients.

Lasso Regression:

Reduces variance and selects features.

ElasticNet:

Balanced approach for correlated/high-dimensional data.

64.11 18.11 Regression Lab Code

Goal:

Download dataset, perform Linear, Ridge, Lasso, Polynomial Regression and check MAE, MSE, RMSE, R2. Explain conclusion.

Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Load dataset
df = pd.read_csv("regression_dataset.csv")

# 2. Basic EDA
print(df.head())
print(df.info())
print(df.isnull().sum())

# 3. Separate X and y
X = df.drop("target", axis=1)
y = df["target"]

# 4. Encode categorical columns
X = pd.get_dummies(X, drop_first=True)

# 5. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Define models
models = {
    "Linear Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LinearRegression())
    ]),
    "Ridge Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", Ridge(alpha=1.0))
    ]),
    "Lasso Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", Lasso(alpha=0.01, max_iter=10000))
    ]),
    "ElasticNet Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", ElasticNet(alpha=0.01, l1_ratio=0.5, max_iter=10000))
    ]),
    "Polynomial Regression": Pipeline([
        ("poly", PolynomialFeatures(degree=2, include_bias=False)),
        ("scaler", StandardScaler()),
        ("model", LinearRegression())
    ])
}
```


7. Evaluate

```
results = []
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    results.append([name, mae, mse, rmse, r2])

result_df = pd.DataFrame(results, columns=["Model", "MAE", "MSE", "RMSE", "R2"])
print(result_df.sort_values("RMSE"))
```

Conclusion format:

The model with lower MAE/RMSE and higher R2 is generally better.

If Polynomial Regression has very low training error but high testing error, it may be
→ overfitting.

If Ridge or Lasso performs better than Linear Regression, regularization improved
→ generalization.

Lasso can be preferred when feature selection is needed.

ElasticNet is useful when correlated features are present.

65 19. Logistic Regression

Logistic Regression is used for classification, not regression.

65.1 19.1 Why Logistic Regression?

Linear Regression predicts continuous values. Classification needs probability between 0 and 1.

Logistic Regression uses sigmoid function.

```
p = 1 / (1 + e^(-z))  
z = wX + b
```

Decision:

```
p >= 0.5 -> class 1  
p < 0.5 -> class 0
```

65.2 19.2 Use Cases

- Loan default prediction
- Spam detection
- Churn prediction
- Disease prediction
- Employee attrition

65.3 19.3 Code

```
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import classification_report  
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import StandardScaler  
  
pipe = Pipeline([  
    ("scaler", StandardScaler()),  
    ("model", LogisticRegression(max_iter=1000))  
)  
  
pipe.fit(X_train, y_train)  
y_pred = pipe.predict(X_test)  
print(classification_report(y_test, y_pred))
```

65.4 19.4 Pros and Cons

Pros:

- Simple.
- Interpretable.
- Good baseline model.
- Outputs probabilities.

Cons:

- Assumes linear decision boundary.
- Sensitive to outliers.
- Needs scaling for regularization.
- Can struggle with complex non-linear patterns.

66 20. Support Vector Machines and Air Quality Lab

66.1 20.1 SVM Basic Principle

SVM finds the best hyperplane that separates classes with maximum margin.

Important terms:

- Hyperplane: decision boundary.
- Margin: distance between hyperplane and nearest points.
- Support vectors: points closest to hyperplane.
- Kernel: transforms data to handle non-linear separation.
- Slack variable: allows margin violations.

66.2 20.2 Linear SVM

Used when classes can be separated by a straight line or plane.

$$w^T x + b = 0$$

Classification:

$$w^T x + b > 0 \rightarrow \text{class } +1$$
$$w^T x + b < 0 \rightarrow \text{class } -1$$

66.3 20.3 Margin Maximization as Geometric Learning

SVM does not just find any boundary. It finds the boundary with maximum margin.

Why maximum margin?

A larger margin usually gives better generalization on unseen data.

66.4 20.4 Soft Margin and Slack Variable

Real data may overlap. Soft margin allows some errors.

Slack:

0: correct and outside margin
between 0 and 1: correct but inside margin
more than 1: misclassified

66.5 20.5 Kernels as Implicit Feature Transformation

Kernel methods allow SVM to behave as if data was transformed into a higher-dimensional space, without explicitly creating those features.

Common kernels:

- Linear
- Polynomial
- RBF
- Sigmoid

Polynomial kernel:

Good for curved boundaries with polynomial relationships.

RBF kernel:

Good for complex non-linear boundaries.

66.6 20.6 Linear vs Non-Linear Decision Boundaries

Linear boundary:

- Easier to interpret.
- Faster.
- Less overfitting risk.
- Works when data is linearly separable.

Non-linear boundary:

- More flexible.
- Can fit complex data.
- Needs careful tuning.
- More overfitting risk.

66.7 20.7 Important SVM Hyperparameters

C: controls margin hardness.

gamma: controls influence of each point in RBF/poly kernels.

kernel: linear, poly, rbf, sigmoid.

degree: degree for polynomial kernel.

High C:

Low training error, smaller margin, more overfitting risk.

Low C:

Larger margin, more regularization, may underfit.

High gamma:

Each point has small influence radius, can overfit.

Low gamma:

Smoother boundary, can underfit.

66.8 20.8 Air Quality Lab: Regression + SVM Classification

Goal:

Download Air Quality dataset from Kaggle.

Predict Air Quality Index using Linear Regression.

Classify it into five categories using SVM: Very Good, Good, Moderate, Poor, Worst.

Common features may include:

- PM2.5
- PM10
- NO2
- SO2
- CO
- O3
- Temperature
- Humidity
- Wind Speed

Step 1: Load and clean data.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVC
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.metrics import classification_report, confusion_matrix, f1_score

# Load dataset
df = pd.read_csv("air_quality.csv")
print(df.head())
print(df.info())
print(df.isnull().sum())

# Drop or fill missing values
df = df.dropna()
```

Step 2: Create AQI category.

```
def aqi_category(aqi):
    if aqi <= 50:
        return "Very Good"
    elif aqi <= 100:
        return "Good"
    elif aqi <= 200:
        return "Moderate"
    elif aqi <= 300:
        return "Poor"
    else:
        return "Worst"

# Assume target numeric AQI column is named AQI
df["AQI_Category"] = df["AQI"].apply(aqi_category)
```

Step 3: Predict AQI using Linear Regression.

```
X = df.drop(["AQI", "AQI_Category"], axis=1)
y_reg = df["AQI"]

X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_reg, test_size=0.2, random_state=42
)

reg_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LinearRegression())
])

reg_model.fit(X_train, y_train)
y_pred = reg_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```

rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2:", r2)

```

Step 4: Classify AQI category using SVM.

```

X_cls = df.drop(["AQI", "AQI_Category"], axis=1)
y_cls = df["AQI_Category"]

X_cls = pd.get_dummies(X_cls, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X_cls, y_cls, test_size=0.2, random_state=42, stratify=y_cls
)

svm_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC(kernel="rbf", C=1.0, gamma="scale", random_state=42))
])

svm_model.fit(X_train, y_train)
y_pred = svm_model.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Weighted F1:", f1_score(y_test, y_pred, average="weighted"))

```

Conclusion format:

Linear Regression predicts numeric AQI and is evaluated using MAE, MSE, RMSE and R2. SVM classifies AQI into categories and is evaluated using confusion matrix, classification report and F1-score. If RBF SVM performs better than linear SVM, the category boundary is likely non-linear. Feature scaling is essential for SVM because it depends on distance/margin geometry.

67 21. Unsupervised Learning Beyond K-Means

67.1 21.1 Hierarchical Clustering

Hierarchical clustering builds a tree of clusters called dendrogram.

Types:

- Agglomerative: bottom-up.
- Divisive: top-down.

Agglomerative steps:

1. Start with each point as its own cluster.
2. Merge closest clusters.
3. Repeat until one cluster remains.
4. Cut dendrogram at chosen height.

Linkage methods:

- Single linkage: minimum distance.
- Complete linkage: maximum distance.
- Average linkage: average distance.
- Ward linkage: minimizes variance.

Code:

```
from sklearn.cluster import AgglomerativeClustering

model = AgglomerativeClustering(n_clusters=5, linkage="ward")
labels = model.fit_predict(X_scaled)
```

67.2 21.2 DBSCAN

DBSCAN is density-based clustering.

Important terms:

- eps: neighborhood radius.
- min_samples: minimum points required to form dense region.
- core point: has enough neighbors.
- border point: near core point but not dense itself.
- noise point: does not belong to cluster.

Advantages:

- Finds irregular shapes.
- Detects noise/outliers.
- Does not require K.

Disadvantages:

- Sensitive to eps and min_samples.
- Struggles with varying density.

Code:

```
from sklearn.cluster import DBSCAN

model = DBSCAN(eps=0.5, min_samples=5)
labels = model.fit_predict(X_scaled)
```

67.3 21.3 Association Rule Mining

Association rules find item relationships.

Example:

```
Bread and Butter -> Milk
```

Important metrics:

Support:

Frequency of itemset in all transactions.

Confidence:

Probability of buying B when A is bought.

Lift:

Lift > 1 means positive association.

67.4 21.4 Anomaly Detection

Anomaly detection finds unusual cases.

Examples:

- Fraud transactions.
- Network attacks.
- Machine failures.
- Medical abnormality.

Isolation Forest code:

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(contamination=0.05, random_state=42)
df["anomaly"] = model.fit_predict(X)

# -1 means anomaly, 1 means normal
print(df["anomaly"].value_counts())
```


68 22. Reinforcement Learning

Reinforcement Learning is learning by interaction.

68.1 22.1 Important Terms

Agent:

Learner or decision maker.

Environment:

World where agent acts.

State:

Current situation.

Action:

Move chosen by agent.

Reward:

Feedback from environment.

Policy:

Strategy used to choose actions.

Value:

Expected long-term reward from a state.

Q-value:

Expected long-term reward for taking an action in a state.

68.2 22.2 Markov Decision Process

MDP defines RL problem with:

- States
- Actions
- Transition probabilities
- Rewards
- Discount factor

Markov property:

Future depends on current state, not full past history.

68.3 22.3 Q-Learning

Q-learning learns action values.

Update idea:

$$\text{New } Q = \text{Old } Q + \text{learning_rate} * (\text{reward} + \text{discount} * \text{best_future_Q} - \text{Old } Q)$$

Explore vs exploit:

Explore: try new actions.
Exploit: use best known action.

69 23. Time Series: Moving Average, Holt, Holt-Winters, ACF, PACF, AR, MA, ARMA, ARIMA

69.1 23.1 What is Time Series?

Time series data is ordered by time.

Examples:

- Daily sales
- Stock price
- Temperature
- Electricity demand
- AQI over time
- Website traffic

Important rule:

Time order matters. Do not randomly shuffle time series data.

69.2 23.2 Components of Time Series

Trend:

Long-term upward or downward movement.

Seasonality:

Pattern repeated at fixed intervals.

Cyclic pattern:

Long-term business/economic cycles, not fixed period.

Noise:

Random variation.

69.3 23.3 Moving Average

Moving average smooths short-term fluctuations.

$MA(t)$ = average of last n observations

Code:

```
df["MA_7"] = df["value"].rolling(window=7).mean()
```

Use:

- Smoothing
- Trend visualization
- Noise reduction

69.4 23.4 Exponential Smoothing

Exponential smoothing gives more weight to recent observations.

$Forecast(t+1) = \alpha * actual(t) + (1-\alpha) * forecast(t)$

High alpha:

More weight to recent data.

Low alpha:

Smoother forecast.

69.5 23.5 Holt's Trend Method

Holt's method handles trend using two components:

- Level
- Trend

Use when data has trend but no seasonality.

69.6 23.6 Holt-Winters Method

Holt-Winters handles:

- Level
- Trend
- Seasonality

Types:

Additive seasonality:

Seasonal variation roughly constant.

Multiplicative seasonality:

Seasonal variation increases/decreases with level.

Code:

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

model = ExponentialSmoothing(
    df["value"],
    trend="add",
    seasonal="add",
    seasonal_periods=12
).fit()

forecast = model.forecast(12)
```

69.7 23.7 Autocorrelation

Autocorrelation measures correlation of a time series with its past values.

Example:

If today's sales are related to yesterday's sales, autocorrelation exists.

Purpose:

- Detect time dependence.
- Select lag features.
- Help identify AR/MA terms.

Lab code:

```

import pandas as pd
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv("time_series.csv", parse_dates=["date"])
df = df.sort_values("date")
df.set_index("date", inplace=True)

# Calculate autocorrelation for lag 1
lag_1_corr = df["value"].autocorr(lag=1)
print("Autocorrelation at lag 1:", lag_1_corr)

# ACF and PACF plots
plot_acf(df["value"].dropna(), lags=30)
plt.show()

plot_pacf(df["value"].dropna(), lags=30)
plt.show()

```

69.8 23.8 ACF and PACF

ACF:

Autocorrelation Function. Shows correlation with different lags.

PACF:

Partial Autocorrelation Function. Shows direct correlation with lag after removing
↪ effect of intermediate lags.

Use:

PACF helps choose p for AR model.
ACF helps choose q for MA model.

69.9 23.9 Stationarity

A stationary series has constant mean and variance over time.

ARIMA generally requires stationarity.

ADF test:

p-value < 0.05 -> stationary
p-value >= 0.05 -> not stationary

Code:

```

from statsmodels.tsa.stattools import adfuller

result = adfuller(df["value"].dropna())
print("ADF Statistic:", result[0])
print("p-value:", result[1])

```

69.10 23.10 Differencing

Differencing removes trend.

```
df["diff_1"] = df["value"].diff()
```

First differencing:

$$Y(t) - Y(t-1)$$

69.11 23.11 AR Model

AR means Auto-Regressive.

Current value depends on past values.

$$Y(t) = c + \phi_1 Y(t-1) + \phi_2 Y(t-2) + \text{error}$$

AR(p): p past lags are used.

69.12 23.12 MA Model

MA means Moving Average model.

Current value depends on past forecast errors.

$$Y(t) = \text{mean} + \text{error}(t) + \theta_1 \text{error}(t-1)$$

MA(q): q past errors are used.

69.13 23.13 ARMA Model

ARMA combines AR and MA.

$$\text{ARMA}(p, q) = \text{AR}(p) + \text{MA}(q)$$

Use when time series is stationary.

69.14 23.14 ARIMA Model

ARIMA means Auto-Regressive Integrated Moving Average.

$$\text{ARIMA}(p, d, q)$$

Where:

- p = AR order
- d = differencing order
- q = MA order

Use ARIMA when series is non-stationary and needs differencing.

69.15 23.15 ARMA vs ARIMA

ARMA: for stationary series.

ARIMA: for non-stationary series after differencing.

69.16 23.16 ARIMA Lab Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Load data
df = pd.read_csv("time_series.csv", parse_dates=["date"])
df = df.sort_values("date")
df.set_index("date", inplace=True)

series = df["value"].dropna()

# Check stationarity
adf_result = adfuller(series)
print("ADF p-value:", adf_result[1])

# Train-test split respecting time order
train_size = int(len(series) * 0.8)
train, test = series.iloc[:train_size], series.iloc[train_size:]

# Fit ARIMA
model = ARIMA(train, order=(1, 1, 1))
fit_model = model.fit()

# Forecast
forecast = fit_model.forecast(steps=len(test))

mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mean_squared_error(test, forecast))

print("MAE:", mae)
print("RMSE:", rmse)

plt.plot(train.index, train, label="Train")
plt.plot(test.index, test, label="Actual")
plt.plot(test.index, forecast, label="Forecast")
plt.legend()
plt.show()

```

Conclusion format:

Autocorrelation helps identify whether past values influence current values.

ARMA is used for stationary series.

ARIMA is used when differencing is required to make the series stationary.

Moving average and exponential smoothing are useful for smoothing and short-term forecasting.

Holt is useful for trend, while Holt-Winters is useful for trend plus seasonality.

70 24. Recommendation Systems

70.1 24.1 What is a Recommendation System?

A recommendation system suggests items to users based on data.

Examples:

- Amazon product recommendations
- Netflix movie recommendations
- YouTube video recommendations
- Flipkart product suggestions
- Spotify song recommendations

70.2 24.2 Data Collection and Storage

Recommendation data may include:

Explicit feedback:

Ratings, likes, reviews, stars.

Implicit feedback:

Clicks, views, watch time, purchases, add-to-cart, search history.

Storage forms:

- User table
- Item table
- Interaction table
- User-item matrix

Interaction table example:

user_id, item_id, rating, timestamp

70.3 24.3 Data Filtering

Filtering removes bad or irrelevant data.

Examples:

- Remove duplicate interactions.
- Remove users with too few interactions.
- Remove items with too few interactions.
- Filter by recent timestamp.
- Normalize ratings.

70.4 24.4 Popularity-Based Recommendation

Recommends most popular items.

Pros:

- Simple.
- Works for new users.

Cons:

- Not personalized.

70.5 24.5 Content-Based Filtering

Uses item features to recommend similar items.

Example:

User watched action movies, recommend more action movies.

Techniques:

- TF-IDF for text.
- Cosine similarity.
- Metadata similarity.

Code idea:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

movies["features"] = movies["genre"] + " " + movies["description"]

vectorizer = TfidfVectorizer(stop_words="english")
tfidf = vectorizer.fit_transform(movies["features"])

similarity = cosine_similarity(tfidf)
```

70.6 24.6 Collaborative Filtering

Collaborative filtering uses user behavior.

Main idea:

Users who behaved similarly in the past may like similar items in future.

Types:

- User-based collaborative filtering
- Item-based collaborative filtering

70.7 24.7 User-Based Collaborative Filtering

Find users similar to target user and recommend what similar users liked.

Example:

User A and User B both liked movies 1, 2, 3.
User B also liked movie 4.
Recommend movie 4 to User A.

70.8 24.8 Item-Based Collaborative Filtering

Find similar items based on user interactions.

Example:

Users who bought laptop also bought laptop bag.
Recommend laptop bag.

70.9 24.9 Matrix Factorization

Matrix factorization decomposes user-item matrix into lower-dimensional user and item embeddings.

User-Item Matrix $\approx$ User Factors x Item Factors

Benefits:

- Handles sparse data.
- Learns hidden features.
- Useful for rating prediction.

Common methods:

- SVD
- ALS
- NMF

70.10 24.10 Hybrid Recommendation

Hybrid systems combine multiple methods.

Example:

Popularity + Content-based + Collaborative Filtering

Benefits:

- Reduces cold start problem.
- Improves personalization.

70.11 24.11 Recommendation Metrics

70.11.1 Precision@K

Out of top K recommended items, how many are relevant?

Precision@K = relevant recommended items in top K / K

70.11.2 Recall@K

Out of all relevant items, how many were recommended in top K?

Recall@K = relevant recommended items in top K / total relevant items

70.11.3 RMSE

Used for rating prediction.

RMSE = error between actual rating and predicted rating

70.11.4 Mean Reciprocal Rank

MRR focuses on the rank of first relevant item.

RR = 1 / rank of first relevant item

MRR = average RR across users

Example:

If first relevant item is at rank 2, RR = 1/2 = 0.5

70.11.5 MAP@K

Mean Average Precision at K evaluates ranking quality across users.

It rewards models that place relevant items earlier.

70.11.6 NDCG

NDCG means Normalized Discounted Cumulative Gain.

It gives higher score when relevant items appear near the top.

Important idea:

Relevant item at rank 1 is more valuable than relevant item at rank 10.

70.12 24.12 Recommendation Lab Skeleton

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

ratings = pd.read_csv("ratings.csv")

# Create user-item matrix
user_item = ratings.pivot_table(
    index="user_id",
    columns="item_id",
    values="rating"
).fillna(0)

# Item similarity
item_similarity = cosine_similarity(user_item.T)
item_similarity_df = pd.DataFrame(
    item_similarity,
    index=user_item.columns,
    columns=user_item.columns
)

# Recommend similar items
def recommend_items(item_id, top_n=5):
    scores = item_similarity_df[item_id].sort_values(ascending=False)
    return scores.iloc[1:top_n+1]

print(recommend_items(item_id=10, top_n=5))
```

70.13 24.13 Cold Start Problem

New user problem:

No history for user.

New item problem:

No interactions for item.

Solutions:

- Use popularity-based recommendations.
- Use content-based features.
- Ask onboarding questions.

- Use demographic or contextual information.

71 25. Introduction to Deep Learning, TensorFlow, Keras, and Neural Networks

71.1 25.1 What is Deep Learning?

Deep Learning is a subset of Machine Learning that uses neural networks with many layers.

It is useful for:

- Image recognition
- Speech recognition
- Natural language processing
- Time series forecasting
- Recommendation systems
- Object detection

71.2 25.2 Why Deep Learning Works

Deep learning performs representation learning.

Instead of manually creating features, neural networks learn useful representations from data.

Example:

In images:
Early layers learn edges.
Middle layers learn shapes.
Deep layers learn objects.

71.3 25.3 Neural Network Structure

Basic neural network:

Input Layer -> Hidden Layer -> Output Layer

Each neuron computes:

$$z = w_1x_1 + w_2x_2 + \dots + b$$
$$a = \text{activation}(z)$$

71.4 25.4 Single Layer Neural Network

A single-layer neural network has input connected directly to output or one computational layer.

Use:

- Foundation for understanding deeper networks.
- Similar to linear/logistic model when no hidden non-linear layers are used.

Limitation:

Cannot learn complex non-linear patterns unless features are transformed.

71.5 25.5 Why Non-Linear Activation Functions are Essential

Without non-linear activation, multiple layers collapse into one linear transformation.

Meaning:

Deep network without activation = linear model.

Non-linear activations allow neural networks to learn complex patterns.

71.6 25.6 TensorFlow

TensorFlow is a deep learning framework used for building and training neural networks.

Features:

- Tensor operations
- Automatic differentiation
- GPU/TPU support
- Deployment tools
- Keras integration

71.7 25.7 Keras

Keras is a high-level neural network API.

It makes model building simple.

Example:

```
import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(16, activation="relu", input_shape=(X_train.shape[1],)),
    keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2)
```

71.8 25.8 Neural Network Applications

- Classification
- Regression
- Image recognition
- Fraud detection
- Speech processing
- Language translation
- Recommendation systems
- Anomaly detection

72 26. Autoencoders

72.1 26.1 What is an Autoencoder?

An autoencoder is a neural network that learns to reconstruct its input.

Structure:

Input -> Encoder -> Latent Representation -> Decoder -> Output

Goal:

Output should be close to input.

72.2 26.2 How Autoencoders Differ from Predictive Models

Predictive model:

Input X -> Target y

Autoencoder:

Input X -> Reconstructed X

Autoencoders are usually unsupervised because they do not need labels.

72.3 26.3 Representation Learning

Encoder compresses input into latent representation.

This representation can capture important features.

Use cases:

- Dimensionality reduction
- Noise removal
- Anomaly detection
- Feature learning
- Image compression

72.4 26.4 Undercomplete Autoencoder

Latent dimension is smaller than input dimension.

Purpose:

Force model to learn compressed representation.

72.5 26.5 Denoising Autoencoder

Input is corrupted/noisy, output is clean.

Purpose:

Learn robust representation and remove noise.

72.6 26.6 Autoencoder Code

```

import tensorflow as tf
from tensorflow import keras

input_dim = X_train.shape[1]

encoder_decoder = keras.Sequential([
    keras.layers.Dense(32, activation="relu", input_shape=(input_dim,)),
    keras.layers.Dense(8, activation="relu", name="latent"),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(input_dim, activation="linear")
])

encoder_decoder.compile(optimizer="adam", loss="mse")
encoder_decoder.fit(
    X_train, X_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)

```

72.7 26.7 Autoencoder for Anomaly Detection

Idea:

Train autoencoder on normal data.
 Normal records reconstruct well.
 Anomalies reconstruct poorly.
 High reconstruction error means anomaly.

73 27. Activation Functions

Activation functions introduce non-linearity.

73.1 27.1 Sigmoid

```
sigmoid(x) = 1 / (1 + e^-x)
```

Output range:

0 to 1

Use:

Binary classification output layer.

Problem:

- Vanishing gradient for very large/small values.

73.2 27.2 Tanh

```
tanh(x) ranges from -1 to 1
```

It is zero-centered, better than sigmoid in hidden layers, but still can suffer from vanishing gradient.

73.3 27.3 ReLU

```
ReLU(x) = max(0, x)
```

Advantages:

- Simple and fast.
- Reduces vanishing gradient problem.
- Common in hidden layers.

Problem:

- Dead ReLU: neurons may output zero permanently.

73.4 27.4 Softmax

Softmax converts raw scores into probabilities for multi-class classification.

Use:

Output layer for multi-class classification.

73.5 27.5 Activation Function Lab

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 200)

sigmoid = 1 / (1 + np.exp(-x))
tanh = np.tanh(x)
relu = np.maximum(0, x)
```

```
plt.plot(x, sigmoid, label="Sigmoid")
plt.legend()
plt.title("Sigmoid")
plt.show()
```

```
plt.plot(x, tanh, label="Tanh")
plt.legend()
plt.title("Tanh")
plt.show()
```

```
plt.plot(x, relu, label="ReLU")
plt.legend()
plt.title("ReLU")
plt.show()
```

Keras activation comparison:

```
from tensorflow import keras
```

```
activations = ["sigmoid", "tanh", "relu"]
```

```
for act in activations:
    model = keras.Sequential([
        keras.layers.Dense(16, activation=act, input_shape=(X_train.shape[1],)),
        keras.layers.Dense(1, activation="sigmoid")
    ])
    model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
    history = model.fit(X_train, y_train, epochs=10, batch_size=32,
        ↪ validation_split=0.2, verbose=0)
    print(act, history.history["val_accuracy"][-1])
```

74 28. Backpropagation and Weight Update with Single Training Example

74.1 28.1 Forward Propagation

Forward propagation calculates prediction.

Steps:

1. Input values enter network.
2. Weighted sum is calculated.
3. Activation function is applied.
4. Output prediction is generated.
5. Loss is calculated.

74.2 28.2 Backpropagation

Backpropagation updates weights by calculating how much each weight contributed to error.

Simple meaning:

Backpropagation is an error-correction mechanism.

Steps:

1. Calculate prediction.
2. Calculate loss.
3. Calculate gradient of loss with respect to weights.
4. Update weights in opposite direction of gradient.

74.3 28.3 Gradient Descent Update Rule

```
new_weight = old_weight - learning_rate * gradient
```

If gradient is positive, weight decreases. If gradient is negative, weight increases.

74.4 28.4 Single Training Example Weight Update

Suppose one neuron:

```
x = 2
w = 0.5
b = 0.1
y_actual = 1
learning_rate = 0.01
```

Forward pass:

```
z = w*x + b = 0.5*2 + 0.1 = 1.1
prediction = sigmoid(1.1)
```

Loss is calculated. Then gradient is calculated. Weight update:

```
w_new = w_old - learning_rate * dLoss/dw
b_new = b_old - learning_rate * dLoss/db
```

This is stochastic learning when done one example at a time.

74.5 28.5 Why Backpropagation is Important

Without backpropagation, neural networks would not know how to improve weights.
It efficiently applies chain rule to update all layers.

74.6 28.6 Common Problems

Vanishing gradient:

Gradients become very small. Early layers learn slowly.

Exploding gradient:

Gradients become very large. Training becomes unstable.

Solutions:

- ReLU activation.
- Batch normalization.
- Proper initialization.
- Gradient clipping.
- LSTM/GRU for sequence models.
- Residual connections in deep networks.

75 29. Deep Learning Essentials: Overfitting, Early Stopping, Dropout, L1/L2

75.1 29.1 Why Deep Networks Overfit

Deep networks have high capacity. If model capacity is greater than data complexity, the model may memorize training data.

Signs of overfitting:

Training loss decreases.
Validation loss starts increasing.
Training accuracy high.
Validation accuracy low.

75.2 29.2 Early Stopping

Early stopping stops training when validation performance stops improving.

It balances underfitting and overfitting.

Code:

```
from tensorflow import keras

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=100,
    callbacks=[early_stop]
)
```

75.3 29.3 Dropout

Dropout randomly turns off some neurons during training.

Purpose:

Prevent neurons from depending too much on each other.

Code:

```
model = keras.Sequential([
    keras.layers.Dense(64, activation="relu", input_shape=(X_train.shape[1],)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(1, activation="sigmoid")
])
```

75.4 29.4 L1 and L2 Regularization

L1 regularization:

Can make weights zero. Useful for feature selection.

L2 regularization:

Shrinks weights. Reduces overfitting.

Keras code:

```
from tensorflow.keras import regularizers

model = keras.Sequential([
    keras.layers.Dense(
        64,
        activation="relu",
        kernel_regularizer=regularizers.l2(0.001),
        input_shape=(X_train.shape[1],)
    ),
    keras.layers.Dense(1, activation="sigmoid")
])
```

75.5 29.5 Regularization as Learning Constraint

Regularization should not be seen only as a post-training fix. It constrains learning during training.

It tells the model:

Learn useful patterns, but do not make weights unnecessarily large or memorize data.

76 30. Batch Training, Mini-Batch, SGD, Momentum, Adam, Gradient Problems

76.1 30.1 Batch Training

Batch training uses the full dataset to calculate one gradient update.

Pros:

- Stable gradient.

Cons:

- Slow for large data.
- Needs more memory.

76.2 30.2 Stochastic Gradient Descent

SGD updates weights using one training example at a time.

Pros:

- Fast updates.
- Can escape local minima due to noise.

Cons:

- Noisy updates.
- Loss curve fluctuates.

76.3 30.3 Mini-Batch Training

Mini-batch uses a small batch of examples for each update.

Common batch sizes:

16, 32, 64, 128

Why popular?

Balance between speed, memory, and stable convergence.

76.4 30.4 Convergence and Efficiency Comparison

Batch Gradient Descent:
Stable but slow.

SGD:
Fast but noisy.

Mini-batch:
Practical balance, most commonly used.

76.5 30.5 Optimizers as Navigation Strategies

Training a neural network means moving on an error surface to find lower loss.

Optimizers decide how to move.

76.6 30.6 Classic Backpropagation with SGD

Uses gradient descent update:

```
weight = weight - learning_rate * gradient
```

76.7 30.7 Momentum

Momentum remembers previous direction and speeds up learning in consistent direction.

Simple idea:

Like a ball rolling downhill.

Benefits:

- Faster convergence.
- Reduces zig-zag movement.

76.8 30.8 Adam Optimizer

Adam means Adaptive Moment Estimation.

It adapts learning rate for each parameter.

Why Adam is popular:

- Works well by default.
- Fast convergence.
- Handles sparse gradients.
- Combines ideas of momentum and adaptive learning rates.

Code:

```
model.compile(  
    optimizer=keras.optimizers.Adam(learning_rate=0.001),  
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

76.9 30.9 Faster Convergence vs Long-Term Generalization

Fast convergence does not always mean best generalization.

Sometimes SGD may generalize better than adaptive optimizers, but Adam is easier and faster in practice.

Practical approach:

Start with Adam.
Tune learning rate.
Use early stopping.
Compare validation performance.

76.10 30.10 Gradient Problems

Vanishing gradient:

- Gradient becomes too small.
- Early layers learn very slowly.
- Common with sigmoid/tanh in deep networks.

Exploding gradient:

- Gradient becomes too large.
- Weights update too much.
- Loss becomes unstable or NaN.

Solutions:

- ReLU or variants.
- Proper initialization.
- Batch normalization.
- Gradient clipping.
- Residual connections.
- LSTM/GRU for sequence problems.

76.11 30.11 Optimizer Lab

```
from tensorflow import keras

optimizers = {
    "SGD": keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "Adam": keras.optimizers.Adam(learning_rate=0.001)
}

for name, opt in optimizers.items():
    model = keras.Sequential([
        keras.layers.Dense(32, activation="relu", input_shape=(X_train.shape[1],)),
        keras.layers.Dense(1, activation="sigmoid")
    ])
    model.compile(optimizer=opt, loss="binary_crossentropy", metrics=["accuracy"])
    history = model.fit(X_train, y_train, epochs=10, batch_size=32,
        ↪ validation_split=0.2, verbose=0)
    print(name, "Validation Accuracy:", history.history["val_accuracy"][-1])
```

77 31. CNN Using PyTorch, PyTorch vs TensorFlow, and Convolution Concept

77.1 31.1 What is PyTorch?

PyTorch is a deep learning framework widely used for experimentation and research-oriented workflows.

Strengths:

- Pythonic style.
- Dynamic computation graph.
- Easy debugging.
- Strong research community.
- Good GPU support.

77.2 31.2 PyTorch vs TensorFlow

PyTorch:

Flexible, easier debugging, popular in research and experimentation.

TensorFlow:

Strong ecosystem, production deployment tools, TensorFlow Serving, TensorFlow Lite.

Keras:

High-level API that makes TensorFlow model building easier.

77.3 31.3 What is CNN?

Convolutional Neural Network is mainly used for image data.

CNN flow:

Image -> Convolution -> Activation -> Pooling -> Flatten -> Dense -> Output

77.4 31.4 Convolution as Representation Learning

Convolution is not just an image-processing trick. It learns useful visual representations.

Early layers learn:

- Edges
- Lines
- Corners

Middle layers learn:

- Shapes
- Textures

Deep layers learn:

- Objects
- Parts of objects

77.5 31.5 Filter / Kernel

A filter is a small matrix that slides over the image and detects patterns.

Example:

A vertical edge filter activates strongly on vertical edges.

77.6 31.6 Stride and Padding

Stride:

How many pixels the filter moves at each step.

Padding:

Adding border pixels around image to control output size.

77.7 31.7 Pooling

Pooling reduces spatial size.

Max pooling:

Takes maximum value from region.

Average pooling:

Takes average value from region.

Benefits:

- Reduces computation.
- Adds small translation invariance.
- Helps control overfitting.

77.8 31.8 Basic PyTorch CNN Code

```
import torch
import torch.nn as nn
import torch.optim as optim

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()
        self.conv_block = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 8 * 8, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.conv_block(x)
        x = self.classifier(x)
        return x
```

```

model = SimpleCNN(num_classes=10)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

77.9 31.9 PyTorch Training Loop Skeleton

```

for epoch in range(10):
    model.train()
    total_loss = 0

    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print("Epoch:", epoch + 1, "Loss:", total_loss / len(train_loader))

```

77.10 31.10 PyTorch Documentation Exploration Lab

Lab tasks:

1. Install PyTorch according to your system.
2. Import torch and print torch version.
3. Create tensor.
4. Perform tensor operations.
5. Build a simple neural network using nn.Module.
6. Load image dataset using torchvision.datasets.
7. Train basic CNN.

Sample tensor code:

```

import torch

x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
y = torch.ones((2, 2))
print(x + y)
print(torch.matmul(x, y))

```

78 32. Transfer Learning, Inception Network, and Data Augmentation

78.1 32.1 What is Transfer Learning?

Transfer learning uses a model trained on a large dataset and adapts it to a new task.

Example:

Use a CNN trained on ImageNet and fine-tune it for medical image classification.

Why useful?

- Saves training time.
- Works with smaller datasets.
- Uses prior visual knowledge.
- Often improves accuracy.

78.2 32.2 Transfer Learning Strategy

Feature extraction:

Freeze pretrained layers and train only final classifier.

Fine-tuning:

Unfreeze some deeper layers and train with small learning rate.

78.3 32.3 Inception Network

Inception network uses multiple filter sizes in parallel.

Idea:

Different objects/patterns may need different receptive fields.

An Inception block may use:

- 1x1 convolution
- 3x3 convolution
- 5x5 convolution
- pooling

Then concatenate outputs.

Benefits:

- Captures multi-scale features.
- Efficient architecture.

78.4 32.4 Why Transfer Learning Helps in Data-Scarce Environments

Training deep CNN from scratch needs large data. Transfer learning uses features already learned from large datasets.

Example:

A pretrained model already knows edges, textures and shapes.
Only final task-specific classifier needs strong adaptation.

78.5 32.5 Data Augmentation

Data augmentation creates modified versions of training images.

Common augmentations:

- Rotation
- Flip
- Crop
- Zoom
- Brightness change
- Contrast change
- Shift

Benefits:

- Increases effective dataset size.
- Reduces overfitting.
- Improves robustness.

PyTorch transform example:

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor()
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])
```

78.6 32.6 Transfer Learning PyTorch Skeleton

```
import torch
import torch.nn as nn
from torchvision import models

num_classes = 2

model = models.resnet18(weights="DEFAULT")

# Freeze pretrained layers
for param in model.parameters():
    param.requires_grad = False

# Replace final layer
in_features = model.fc.in_features
model.fc = nn.Linear(in_features, num_classes)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.fc.parameters(), lr=0.001)
```

Conclusion:

Transfer learning is useful when dataset is small and task is related to visual
↪ patterns learned by pretrained models.

Data augmentation improves generalization by exposing model to variations.

79 33. Object Detection and YOLO Algorithm

79.1 33.1 Image Classification vs Object Detection

Image classification:

Predicts one label for whole image.
Example: image contains dog.

Object detection:

Finds object location and class.
Example: dog at bounding box coordinates.

Object detection is more complex because model must predict:

- What object is present.
- Where object is located.
- How many objects exist.
- Confidence score.

79.2 33.2 Object Detection Output

Typical output:

```
class label  
confidence score  
bounding box: x_min, y_min, x_max, y_max
```

79.3 33.3 Important Metrics for Object Detection

IoU, Intersection over Union:

$\text{IoU} = \text{area of overlap} / \text{area of union}$

mAP, mean Average Precision:

Average precision across classes and IoU thresholds.

79.4 33.4 What is YOLO?

YOLO means You Only Look Once.

Main idea:

YOLO reframes object detection as a single-step prediction problem.

Traditional detection methods may use multiple stages. YOLO predicts bounding boxes and class probabilities in one forward pass.

79.5 33.5 Why YOLO is Fast

YOLO processes the full image once and directly predicts detections.

Advantages:

- Fast inference.
- Suitable for real-time detection.
- End-to-end detection.

Limitations:

- Small object detection can be challenging.
- Needs labelled bounding box data.
- Accuracy depends on dataset and training quality.

79.6 33.6 YOLO High-Level Working

1. Input image is resized.
2. Image passes through CNN backbone.
3. Feature maps are generated.
4. Detection head predicts boxes, confidence, and classes.
5. Non-Max Suppression removes duplicate boxes.
6. Final detections are returned.

79.7 33.7 Non-Max Suppression

If multiple boxes detect same object, NMS keeps the best box and removes overlapping lower-confidence boxes.

79.8 33.8 YOLO Lab Workflow

Syllabus says implement YOLO algorithm. For lab, high-level implementation can use a pretrained YOLO model or train/fine-tune on custom data.

Common workflow:

1. Install required YOLO package/framework.
2. Prepare dataset with images and labels.
3. Use pretrained weights.
4. Run prediction on sample images.
5. Train/fine-tune if dataset is available.
6. Evaluate detections using confidence, IoU, mAP.

Generic prediction-style code:

```
# Example style only. Actual package commands may vary by environment.
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model.predict(source="test_image.jpg", conf=0.25)

for result in results:
    result.show()
```

Training-style workflow:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
model.train(data="data.yaml", epochs=10, imgsz=640)
```

Dataset YAML concept:

```
train: path/to/train/images
val: path/to/val/images
nc: 2
names: ["class1", "class2"]
```

Conclusion:

YOLO is used for real-time object detection because it predicts bounding boxes and
→ class probabilities in a single pass.
Object detection is harder than classification because it requires both class
→ prediction and localization.

80 34. Complete Lab Code Bank and Exam Patterns

80.1 34.1 Basic ML Lab Template

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
    f1_score

# Load
DATA_PATH = "dataset.csv"
df = pd.read_csv(DATA_PATH)

# Inspect
print(df.head())
print(df.tail())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())

# Clean
for col in df.select_dtypes(include="number").columns:
    df[col] = df[col].fillna(df[col].median())

for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].fillna(df[col].mode()[0])

# Split X/y
X = df.drop("target", axis=1)
y = df["target"]

# Encode
X = pd.get_dummies(X, drop_first=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

80.2 34.2 Classification Model Comparison Template

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score, classification_report

models = {
    "Logistic Regression": Pipeline([
```

```

        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=1000))
    ]),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "KNN": Pipeline([
        ("scaler", StandardScaler()),
        ("model", KNeighborsClassifier(n_neighbors=5))
    ]),
    "Naive Bayes": GaussianNB(),
    "SVM": Pipeline([
        ("scaler", StandardScaler()),
        ("model", SVC(kernel="rbf", random_state=42))
    ])
}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print("\nModel:", name)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("F1:", f1_score(y_test, y_pred, average="weighted"))
    print(classification_report(y_test, y_pred))

```

80.3 34.3 Complete Pipeline with ColumnTransformer

```

import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X = df.drop("target", axis=1)
y = df["target"]

numeric_cols = X.select_dtypes(include="number").columns
categorical_cols = X.select_dtypes(include="object").columns

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_cols),
    ("cat", categorical_pipe, categorical_cols)
])

```

```

model = Pipeline([
    ("preprocess", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))

```

80.4 34.4 Save and Load Model

```

import joblib

joblib.dump(model, "final_model.pkl")
loaded_model = joblib.load("final_model.pkl")
predictions = loaded_model.predict(X_test)

```

81 35. Final Revision Tables

81.1 35.1 Algorithm Selection

Linear relationship and continuous target -> Linear Regression
Non-linear continuous target -> Polynomial Regression, Random Forest Regressor,
↳ XGBoost Regressor
Binary classification -> Logistic Regression, SVM, Random Forest
High-dimensional text classification -> Naive Bayes, Logistic Regression
Small high-dimensional data -> SVM
Large tabular structured data -> Random Forest, XGBoost, LightGBM, CatBoost
Customer segmentation -> K-Means, Hierarchical, DBSCAN
Feature reduction -> PCA, Kernel PCA, Random Projection
Market basket -> Apriori, Association Rules
Recommendation -> Collaborative Filtering, Matrix Factorization, Hybrid
Time series -> Moving Average, Holt-Winters, ARIMA
Images -> CNN, Transfer Learning
Object detection -> YOLO
Sequence data -> RNN, LSTM, GRU

81.2 35.2 Scaling Required or Not

Scaling required:

KNN, SVM, Logistic Regression, Linear Regression with regularization, K-Means, PCA,
↳ Neural Networks

Scaling usually not required:

Decision Tree, Random Forest, XGBoost, LightGBM, CatBoost

81.3 35.3 Overfitting Control

Linear/Ridge/Lasso: regularization
Decision Tree: pruning, max_depth, min_samples_leaf
Random Forest: more trees, max_depth, min_samples_leaf
Boosting: learning_rate, n_estimators, max_depth, subsampling
SVM: C and gamma tuning
Neural Network: dropout, early stopping, L1/L2, data augmentation
CNN: data augmentation, transfer learning, dropout

81.4 35.4 Most Important Interview Lines

Machine Learning learns patterns from data without explicit programming.
Supervised learning uses labelled data.
Unsupervised learning finds hidden patterns without labels.
Reinforcement learning learns through rewards and punishments.
Regression predicts continuous values.
Classification predicts categories.
Clustering groups similar data points.
PCA reduces dimensions by preserving maximum variance.
Regularization reduces overfitting by penalizing model complexity.
Cross-validation gives more reliable evaluation than one train-test split.
Random Forest reduces variance using many decision trees.
Boosting builds models sequentially to correct previous errors.

SVM finds maximum margin hyperplane.
Naive Bayes applies Bayes theorem with independence assumption.
KNN predicts based on nearest examples.
ARIMA is used for time series forecasting.
Recommendation systems suggest items using user/item interactions.
Deep learning learns representations using neural networks.
CNN is best for image data.
YOLO performs object detection in one forward pass.

82 36. Final CDAC Study Order

Study in this order for best understanding and lab readiness:

1. AI, ML, DL basics.
2. ML lifecycle.
3. Supervised, unsupervised, reinforcement learning.
4. Data preprocessing, encoding, scaling, train-test split.
5. Information theory, entropy, information gain.
6. PAC learning, VC dimension, SRM, Occam, No Free Lunch.
7. Regression: Linear, Polynomial, Ridge, Lasso, ElasticNet.
8. Regression metrics: MAE, MSE, RMSE, R2.
9. Logistic Regression.
10. Classification metrics: Accuracy, Precision, Recall, F1, ROC-AUC, LogLoss.
11. Decision Tree and CART.
12. Random Forest.
13. GBM, XGBoost, LightGBM, CatBoost, Stacking.
14. SVM: linear, polynomial, RBF.
15. Naive Bayes and probability calculation.
16. LDA and QDA.
17. KNN.
18. K-Means, WSS, Elbow, Silhouette.
19. Hierarchical clustering and DBSCAN.
20. PCA, Kernel PCA, Random Projection.
21. Association Rule Mining.
22. Recommendation Systems and ranking metrics.
23. Time Series: Moving Average, Holt, Holt-Winters, ACF, PACF, AR, MA, ARMA, ARIMA.
24. Reinforcement Learning basics.
25. Deep Learning basics.
26. TensorFlow and Keras.
27. Autoencoders.
28. Activation functions.
29. Backpropagation and weight updates.
30. Early stopping, dropout, L1/L2.
31. Batch, mini-batch, SGD, Momentum, Adam.
32. PyTorch.
33. CNN.
34. Transfer Learning, Inception, Data Augmentation.
35. Object Detection and YOLO.
36. Complete lab practice and model comparison.

83 End Note

This version is designed as a single revision and lab-preparation PDF. Use the theory sections for concepts, the lab sections for practical implementation, and the final revision tables before exams/interviews.